Manzano et al. *EPJ Quantum Technology* (2025) 12:28
https://doi.org/10.1140/epjqt/s40507-025-00328-3

EPJ Quantum Technology
a SpringerOpen Journal

RESEARCH Open Access

Alternative pipeline for option pricing using quantum computers

Alberto Manzano[1*], Gonzalo Ferro[2], Álvaro Leitao[1], Carlos Vázquez[1] and Andrés Gómez[2]

*Correspondence: alberto.manzano.herrero@gmail.com
[1]Department of Mathematics and CITIC, University of A Coruña, Campus Elviña s/n, A Coruña, 15071, A Coruña, Spain
Full list of author information is available at the end of the article

Abstract

In this work we present an alternative methodology to the standard Quantum Accelerated Monte Carlo (QAMC) applied to derivatives pricing. Our pipeline benefits from the combination of a new encoding protocol, referred to as the direct encoding, and an amplitude estimation algorithm, the modified Real Quantum Amplitude Estimation (mRQAE) algorithm. On the one hand, the direct encoding prepares a quantum state which contains the information about the sign of the expected payoff. On the other hand, the mRQAE is able to read all the information contained in the quantum state. Although the procedure we describe is different from the standard one, the main building blocks are almost the same. Thus, all the extensive research that has been performed is still applicable. Moreover, we experimentally compare the performance of the proposed methodology against the standard QAMC employing a quantum emulator and show that we retain the speedups.

Keywords: Option pricing; Quantum Accelerated Monte Carlo; Amplitud Estimation

1 Introduction

Over the last few years, there has been an increasing interest in the application of quantum computing to quantitative finance. One of the reasons for such boost of popularity is because many algorithms used by financial institutions demand a high computing capacity and quantum computing promises relevant speedups in some relevant cases. Among the different financial applications that could benefit from the use of quantum computing, we focus on the pricing of financial derivatives. We refer the reader to [1] for a recent survey of classical and quantum techniques for option pricing.

As it is well known in the literature (see for instance [2]), the pricing of financial derivatives can be formulated in terms of the computation of the expectation of the derivatives payoff with respect to a given probability measure. This computation can be very consuming in terms of computational resources and is typically performed by means of Monte Carlo (MC) methods. In the context of classical Monte Carlo (CMC) methods, the quantum computing community has proposed a quantum version which can obtain quadratic speedups for very general settings, as indicated in [3]. We will refer to such techniques as Quantum Accelerated Monte Carlo (QAMC). They have been successfully applied to problems of financial derivatives pricing (see [4, 5] for the first seminal approaches, and [6–8] for more recent contributions to the field).

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

However, to the best of our knowledge, in the setups treated in the literature it is assumed that the price and the payoff of the derivative are strictly positive. If it is not the case, in order for the technique to work, we need to divide the problem into two: an estimation of the derivatives price where the codomain is positive and a separate estimation of the derivatives price where the codomain is negative. Then, both solutions are combined to obtain the final estimate. This article is centered around the idea of building an alternative protocol which avoids the hassle of separating into codomains.

There are three main contributions in this article. First, the direct encoding protocol, which allows to load into the quantum circuit information about the sign of the expected payoff. Second, the modified real quantum amplitude estimation (mRQAE) which is an asymptotically optimal version of the real quantum amplitude estimation (RQAE) with the feature of recovering the sign of the underlying amplitude (see [9]). Third, the whole pipeline, which exploits the synergies between both techniques.

The manuscript is organised as follows. In Sect. 2 we begin by revising the main building components of CMC and QAMC. More precisely, Sects. 2.1 and 2.2 contain a brief summary of the classical and quantum techniques to tackle the derivative pricing problem through Monte Carlo like techniques. Then, Sect. 3 is devoted to the introduction of the new proposed strategy to perform a quantum Monte Carlo. The first part, in Sect. 3.1, presents the direct encoding for negative payoffs along with its empirical evaluation. The second part, in Sect. 3.2, leverages the power of the mRQAE algorithm (see Appendix B) for the pricing of derivative contracts with negative prices. Once again we perform an empirical assessment. In Sect. 4 we summarize the main conclusions.

2 Preliminaries

In financial markets, a derivative is a financial contract whose value depends on the future performance of one or several assets usually referred to as underlying assets, or simply underlying (see, for example [10]). In this context, derivatives pricing consists of obtaining the price of the derivative at any previous date to maturity date. For this purpose, the uncertain future dynamics of the price of the underlying asset must be taken into account, which is usually modelled in terms of stochastic differential equations.

More precisely, if we denote by S_t the price of the underlying asset at time t, a general Itô process that satisfies the following Stochastic Differential Equation (SDE) can be considered:

$$dS_t = \alpha(t, S_t)\, dt + \beta(t, S_t)\, dW_t, \tag{1}$$

where α and β are real functions that represent the drift and the diffusion to be specified for the particular model, while W_t denotes a Brownian motion in a particular probability space, so that W_t follows a $\mathcal{N}(0, t)$ distribution (and its increment dW_t follows a $\mathcal{N}(0, dt)$).

Another ingredient in the pricing of derivatives is the payoff. We denote by V_t the price of the derivative at time $t \in [0, T]$, where T is the maturity date.

In the present work, on the one hand we will present several examples corresponding to derivatives with payoffs only depending on the asset price at maturity. More precisely, we consider call and put vanilla and digital options, jointly with a derivative with linear payoff representing the case with non-positive price and payoff. On

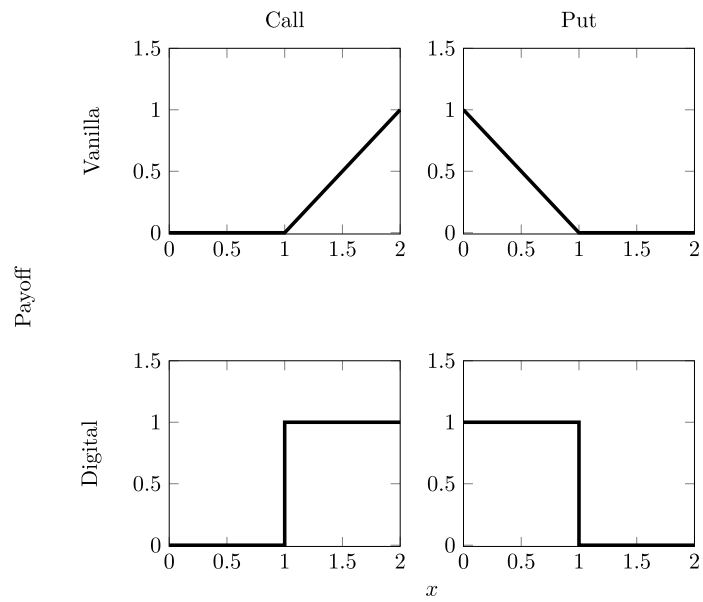


Figure 1 Payoff functions for digital and vanilla options with $K = 1$

the other hand, we will consider a more complex derivative with path-dependent payoff, which corresponds to a cliquet option, that also illustrates the case of non-positive price.

For the case of derivatives whose payoff only depends on the value of the asset at maturity, we have that $V_T = F(S_T)$, where F represents the payoff function. If we denote the strike price by K, the nonlinear payoff functions that we consider in this article are (see also Fig. 1):

- Vanilla call option: $F(x) = \max(x - K, 0)$.
- Digital call option: $F(x) = 1_{x>K}$.
- Vanilla put option: $F(x) = \max(K - x, 0)$.
- Digital put option: $F(x) = 1_{x<K}$.

Moreover, we consider a derivative with linear payoff $F(x) = x - K$ (see Fig. 2).

The expression of the payoff of the path-dependent cliquet option is more complex (see [11], for example). In order to define the cliquet option payoff, for the maturity date T, let us consider the set of dates $T_0 = 0 < T_1 < \cdots < T_{N-1} < T_N = T$, which are referred to as reset days and that divide the time interval $[0, T]$ into N subintervals referred to as reset periods of length $\Delta T_n = T_n - T_{n-1}$, $n = 1, \ldots, N$. If we consider an asset with price S_t at time t, the return of the asset over the period $[T_{n-1}, T_n]$ is given by:

$$R_n = \frac{S_{T_n}}{S_{T_{n-1}}} - 1$$

If we denote by c and f the cap and floor levels for the returns, with $f < c$, the truncated returns are given by:

$$TR_n = \max(\min(R_n, c), f).$$

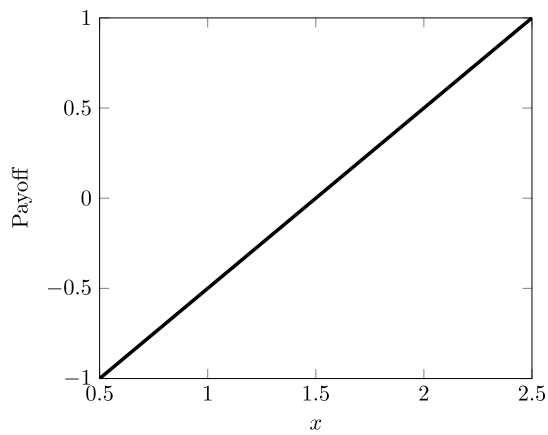


Figure 2 Linear payoff function with $K = 1.5$

In terms of the truncated returns, for a normalized notional amount equal to one, the payoff of the general cliquet option is defined as:

$$V_T = \min\left(\max\left(\sum_{n=1}^{N} TR_n, f_g\right), c_g\right), \tag{2}$$

where c_g and f_g denote the levels of global cap and floor, respectively. In the definition of parameters, the condition $Nf < f_g < c_g < Nc$ is usually satisfied.

By using mathematical finance tools, mainly martingale properties, Itô's lemma and the Feynmann-Kàc theorem, the following expression for price of the derivative at time t, V_t, can be obtained (see, for example [10]):

$$V_t = e^{-r(T-t)}\mathbb{E}^Q[V_T|\mathcal{F}_t], \tag{3}$$

where $\mathbb{E}^Q$ denotes the expectation under the risk neutral measure Q, r is the constant risk-free interest rate, V_T is the payoff of the derivative and $\mathcal{F}_t$ denotes the σ-algebra containing the information until time t. In this way, expression (3) indicates that the value of the derivative is the discounted price of the expected value of the payoff, conditioned to the current information of the market.

Note that the payoff V_T is given and can depend only on the value of S_T or on the values of the asset at the reset dates in the case of the cliquet option. In order to cover both cases, we can write $V_T = F(S)$, where S represents the whole history of asset prices.

In view of the pricing expression (3), the valuation of these financial derivatives mainly requires the computation of the involved expectation. Next, we briefly introduce one of the most popular techniques for computing such expectation, namely the Monte Carlo method. We revise the Classical Monte Carlo (CMC) in Sect. 2.1 and the Quantum Accelerated Monte Carlo (QAMC) in Sect. 2.2.

2.1 Classical Monte Carlo for derivatives pricing

The CMC method for derivatives pricing in finance is composed of two steps:

1. Simulation of sample paths of the underlying asset by means of the numerical solution of the SDE (1).
2. Use of Monte Carlo integration to compute the expectation that appears in expression (3).

2.1.1 Simulation of sample paths of the underlying price

For the simulation of the sample paths followed by the underlying asset price, there exist several numerical methods for solving the associated SDE (1). In order to illustrate the whole procedure we will use hereon the Euler-Maruyama method as an example. The application of the Euler-Maruyama scheme to the general SDE (1) leads to the expression [12]:

$$S_{t_{j+1}} = S_{t_j} + \alpha(t_j, S_{t_j})\Delta t + \beta(t_j, S_{t_j})\sqrt{\Delta t}Z, \tag{4}$$

where Z is the standard normal random variable, i.e. with mean equal to 0 and variance equal to 1, and Δt is the time step that is considered in the numerical method. Moreover, $\Delta t = (T - t)/M$, M being the number of time steps, T the maturity date, t the initial time and $t_j = t + j\Delta t$ with $j = 0, \ldots, M$. By using Equation (4), it is straightforward to produce samples of S_T, starting from a value of S_t at time t. More precisely, we proceed as follows:

- Start with the initial point S_t.
- Draw a sample of the standard normal random variable Z.
- Compute a sample of $S_{t+\Delta t}$ from the random sample generated in the previous step.
- Repeat the previous process, starting from the last calculated value, until a sample of S_T is obtained.

2.1.2 Integration by Monte Carlo

If we repeat N_C times the procedure described in Sect. 2.1.1, we obtain a set of N_C paths of the price evolution and samples S_T^i, $i = 1, \ldots, N_C$ of the random variable S_T, which can be then used to estimate the expectation that appears in expression (3) as follows:

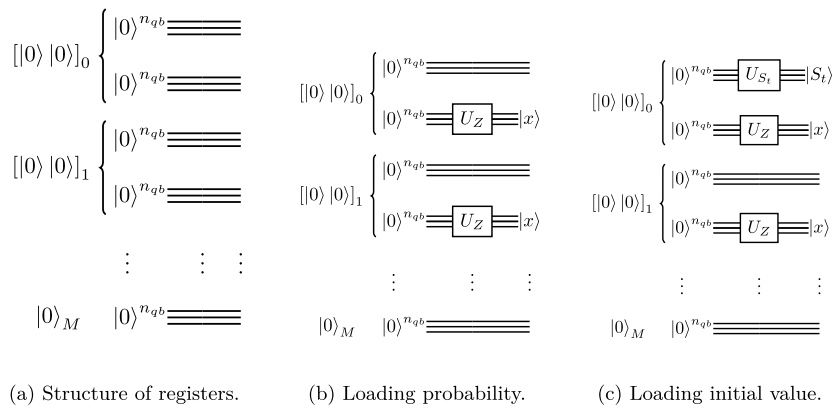
$$\mathbb{E}[V_T|\mathcal{F}_t] = \frac{1}{N_C}\sum_{i=1}^{N_C} V_T^i + \epsilon_{\text{EM}} + \epsilon_{\text{CMC}}. \tag{5}$$

In expression (5), N_C is the number of paths S^i generated by numerically solving the SDE, which are required to compute the corresponding payoffs $V_T^i = F(S^i)$ in each derivative contract. Moreover, ϵ_{CMC} is the statistical error due to the Monte Carlo approximation of the expectation and ϵ_{EM} is the error coming from the discretization of the SDE, *i.e.,* the error due to the Euler-Maruyama scheme. The statistical error ϵ_{CMC} scales as [2]:

$$\epsilon_{\text{CMC}} \sim \frac{1}{\sqrt{N_C}}. \tag{6}$$

The order of the error due to the Euler-Maruyama scheme is [13]:

$$\epsilon_{\text{EM}} \sim \Delta t \sim \frac{1}{M}. \tag{7}$$



(a) Structure of registers. (b) Loading probability. (c) Loading initial value.

Figure 3 Circuit initialisation

2.2 Quantum accelerated Monte Carlo for derivatives pricing

The QAMC for pricing contains three main ingredients:

- A quantum circuit which samples paths with the same probability as the classical circuit.
- An operator which encodes the payoff of the specific derivative contract into the quantum state.
- An amplitude estimation routine, which allows to retrieve the quantity of interest from the amplitude of a quantum state (and produces the actual speedup).

In Sect. 2.2.1 we briefly discuss the first issue, while we reserve Sect. 2.2.2 for the second and third issues.

2.2.1 Quantum simulation

The QAMC algorithm begins by creating a state in superposition where the probabilities of each path match those of the classical process discretized by using some numerical scheme such as the Euler-Maruyama formula. Alternative methods for the numerical solution of SDEs with different orders of convergence can be considered (see, for example [13]). In order to build the algorithm, $M+1$ different registers are needed, one per time step. The first M registers are composed of two registers of n_{qb} qubits each (see Fig. 3a):

$$[|0\rangle|0\rangle]_0 \otimes [|0\rangle|0\rangle]_1 \otimes \cdots \otimes |0\rangle_M, \tag{8}$$

where $[|0\rangle|0\rangle]_m = \left[|0\rangle^{\otimes n_{qb}}|0\rangle^{\otimes n_{qb}}\right]_m$, $m = 0,\ldots,M$ and $|0\rangle_M = |0\rangle^{\otimes n_{qb}}$.

Each of the individual registers $|0\rangle^{\otimes n_{qb}}$ will be used to represent a decimal number. For simplicity, it can be understood as a single precision register. In order to generate a state in superposition which matches the probabilities of each path defined by the Equation (4), we need a standard normal sample generator. In the QAMC algorithm, this generator is represented by the unitary operator U_Z which performs the following transformation:

$$U_Z|0\rangle|0\rangle = \sum_{j=1}^{J}\sqrt{p_Z(x_j)}|0\rangle|x_j\rangle = |0\rangle|x\rangle, \tag{9}$$

where x is a set of J numbers that can be represented by the n_{qb} qubits from the individual registers and $p_Z(x)$ is a discretized version of the standard normal probability distribution defined in the set of points $x = \{x_1, x_2, \ldots, x_J\}$. Note that, in general, J does not need to be equal to $2^{n_{qb}}$. The efficiency of the transformation (9) is crucial for the overall efficiency of the algorithm. In the best case, this efficiency can be achieved using $O(\log_2(J))$ gates (see [14]). In the worst case, it can be achieved in $O(J\log_2(J))$ combining the results in [14] and [15].

The first step requires applying the operator U_Z to one of the members of all pairs $[|0\rangle|0\rangle]_i$, thus obtaining the state:

$$\begin{aligned}&[U_Z|0\rangle|0\rangle]_0 \otimes [U_Z|0\rangle|0\rangle]_1 \otimes \cdots \otimes |0\rangle_M \\ &= \left[\sum_{j=1}^{J}\sqrt{p_Z(x_j)}|0\rangle|x_j\rangle\right]_0 \otimes \left[\sum_{j=1}^{J}\sqrt{p_Z(x_j)}|0\rangle|x_j\rangle\right]_1 \otimes \cdots \otimes |0\rangle_M.\end{aligned}$$

In this configuration, the amplitudes encode the square root of the probabilities for all the different combinations of x in the different steps. Next, the left register in the pair $[|0\rangle|0\rangle]_0$ has to be initialised to $|S_t\rangle$:

$$\left[U_{S_t}|0\rangle|y\rangle\right]_0 = \left[|S_t\rangle|y\rangle\right]_0, \quad \forall\, |y\rangle. \tag{10}$$

Figure 3 depicts schematically this process. Once the circuit is correctly initialised, an evolution operator must be applied. This evolution operator $U_{\Delta t}$ acts upon three individual registers as follows:

$$U_{\Delta t}\,[|S_{t+m\Delta t}\rangle|x\rangle]_m\,[|0\rangle]_{m+1} \longrightarrow [|S_{t+m\Delta t}\rangle|x\rangle]_m\left[|S_{t+(m+1)\Delta t}(S_{t+m\Delta t}, x)\rangle\right]_{m+1}, \tag{11}$$

where the update rule is given, for instance, by Equation (4). Note that other update rules can be used instead. After repeatedly applying the operator $U_{\Delta t}$ defined in Equation (11), the final quantum state $|S\rangle$:

$$|S\rangle := U_S|0\rangle = \sum_{k=0}^{\mathcal{K}-1}\sqrt{p_S(S_k)}|S_k\rangle, \tag{12}$$

where M is the number of time steps, $\mathcal{K} = J^M$ are the number of possible paths defined by the given (space and time) discretization and $p_S(S_k)$ is the probability of generating the path S_k. Figure 4 depicts schematically this process. So far, a quantum circuit has been built which samples paths with the same probability as the classical circuit does. Moreover, the computational cost of one execution of the circuit is equivalent to one execution of the classical circuit, *i.e.*, the number of gates needed to sample one path from the classical and the quantum circuit is "the same", since the classical circuit can always be translated to a quantum one using Toffoli gates (see [16], for example). However, note that classical and quantum gates are not directly comparable.

2.2.2 Amplitude estimation

As discussed in the previous section, up to this point the quantum and the classical circuit have the same complexity. Nevertheless, when the error correction is taken into consider-

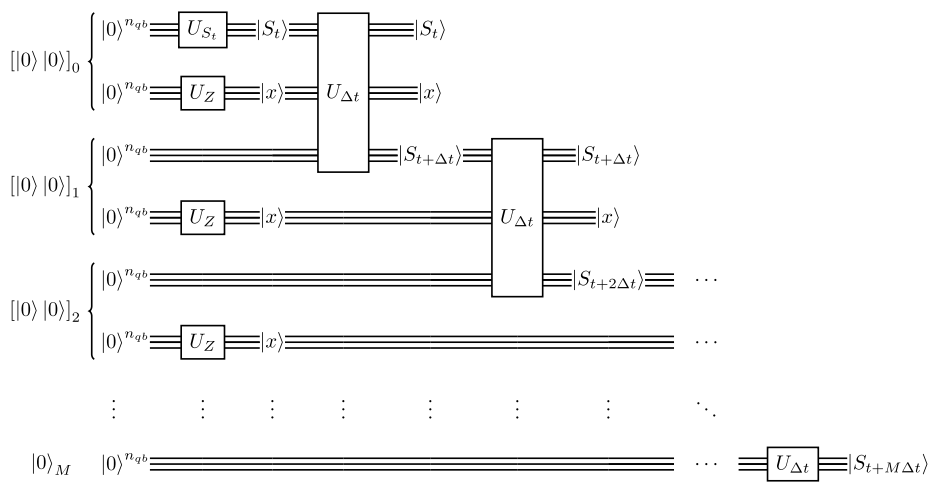


Figure 4 Sketch description of the construction of the oracle defined in Equation (12)

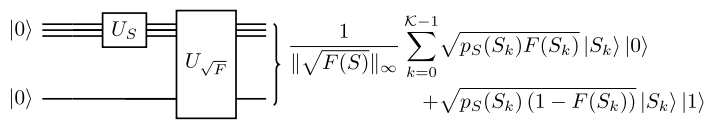


Figure 5 Scheme of the generation of the oracle in the square root encoding. The gate U_S corresponds to Equation (12). The gate $U_{\sqrt{F}}$ corresponds to Equation (13)

ation, the current quantum gates are much slower than the analogous classical ones. Next, the mechanism that produces a speedup is briefly detailed.

Once the state $|S\rangle$ in Equation (12) is generated, the next step is to define the operator $U_{\sqrt{F}}$ such that pushes the square root of the derivatives payoff F into the amplitude. For this reason, we will call this way of encoding *square root encoding*. For this purpose, an additional single qubit register is needed:

$$\begin{aligned}|\sqrt{F}\rangle = U_{\sqrt{F}}|S\rangle|0\rangle = &\frac{1}{\|\sqrt{F(S)}\|_\infty}\sum_{k=0}^{\mathcal{K}-1}\sqrt{p_S(S_k)F(S_k)}|S_k\rangle|0\rangle \\ &+\sqrt{p_S(S_k)\,(1-F(S_k))}|S_k\rangle|1\rangle.\end{aligned} \tag{13}$$

Moreover, it is tacitly assumed that the operator $U_{\sqrt{F}}$ can be efficiently implemented. Figure 5 depicts schematically the overall process. We will denote by U_{SRE} the combination of the path generating oracle U_S and the payoff oracle $U_{\sqrt{F}}$:

$$U_{\mathrm{SRE}} := U_{\sqrt{F}}U_S. \tag{14}$$

The probability of measuring zero in the rightmost register after the application of U_{SRE} is given by:

$$P_{|0\rangle} = \frac{1}{\|\sqrt{F(S)}\|_\infty^2}\sum_{k=0}^{\mathcal{K}-1}|p_S(S_k)F(S_k)|\,. \tag{15}$$

Table 1 Comparison of the order of the errors for the CMC and the QAMC

	Error
CMC	$O(1/\sqrt{N_C})$
QAMC	$O(1/N_Q)$

Hence, getting an estimation $\widetilde{P}_{|0\rangle}$ of $P_{|0\rangle}$ yields an estimation of the expectation in Equation (3) except for the normalization constants:

$$\mathbb{E}[F(S_T)|\mathcal{F}_t] = \sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k) + \epsilon_{\mathcal{K}} + \epsilon_{\mathrm{EM}} \approx \|\sqrt{F(S)}\|_\infty^2 \widetilde{P}_{|0\rangle} + \epsilon_{\mathcal{K}} + \epsilon_{\mathrm{EM}} + \epsilon_{\mathrm{QAMC}}, \quad (16)$$

where $\sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k)$ is the discretized expectation, ϵ_{EM} is same Euler-Maruyama error as in Equation (5), $\epsilon_{\mathcal{K}}$ is the discretization error, $\widetilde{P}_{|0\rangle}$ is an estimation of the probability of obtaining zero in the last register and ϵ_{QAMC} is the sampling error given by,

$$\epsilon_{\mathrm{QAMC}} = \left| \sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k) - \|\sqrt{F(S)}\|_\infty^2 \widetilde{P}_{|0\rangle} \right|. \quad (17)$$

The discretization error $\epsilon_{\mathcal{K}}$ is intimately related with the discretization of the standard normal probability distribution. When we use a fine-grain discretization this error can be considered negligible compared with the other ones. This is the case of the CMC, where we typically use 32 bits (single precision) to represent the standard normal. For this reason and to avoid confusion we have omitted any reference to the discretization error in Equation (5).

By using amplitude estimation techniques, we know that the sampling error ϵ_{QAMC} is of order [17]:

$$\epsilon_{\mathrm{QAMC}} \sim \frac{1}{N_Q}, \quad (18)$$

with N_Q being the number of calls to the oracle defined by Equation (14). Recall that this oracle is strictly the same as in the classical algorithm described in Sect. 2.1.1. Thus, each call to the oracle U_{SRE} is equivalent to M steps of the Euler-Maruyama scheme. Technically speaking, since the application of amplitude estimation techniques requires the use of the adjoint of the oracle U_{SRE} we are assuming that both have the same cost.

In Table 1 we show the computational cost of CMC and QAMC, measured in terms of the number of queries. It can be easily seen that the QAMC performs quadratically better than the CMC. Moreover, the same scaling applies when we increase the number of dimensions.

2.2.3 QAMC simplifications and practical implementation

So far, we have described the general setup of QAMC for pricing. A rough estimation indicates that we would require the order of hundreds or thousands of logical qubits to build the algorithm with single precision registers and a few time steps. With the current hardware, this is not feasible (see [18]). Hence, in order to conceptually test this technique, we need to perform several simplifications.

The first simplification would be reducing as much as possible the number of time steps. In the limit, when we have payoffs depending only on the underlying price at maturity, we could perform a single time step. Nevertheless, in numerical schemes such as Euler-Maruyama, a very big time step produces a very big error. For that reason, we restrict ourselves to models where we know how to do exact simulation, thus avoiding the need of doing several steps. This happens when we can obtain the exact solution of the governing SDE. Thus, in this work, we will consider the classical (and well known) model given by the following Black-Scholes SDE under the risk neutral measure [19]

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t, \tag{19}$$

where r denotes the risk-free rate and σ is the volatility of the underlying asset price. As the expression of the exact solution of SDE (19) is known, starting for a given value S_t, the simulation of the random variable S_T under the risk-neutral measure can be exactly carried out in one step by:

$$S_T = S_t \exp\left(\left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma Z\sqrt{T-t}\right). \tag{20}$$

In yet another simplification, we assume that, instead of having a unitary operator U_Z which encodes the standard normal, we have an analogous unitary U_{BS} which encodes the Black-Scholes path distribution p_{BS}. With this last simplification, we only need a single register to perform the whole simulation. Note that, regardless of all the simplifications, the algorithm is conceptually the same: we have a quantum circuit specified by the oracle $U_S = U_{\mathrm{BS}}$ which generates samples for the underlying price at maturity, S_T, with the correct probability distribution.

Throughout the manuscript, we will show different numerical experiments, all of them performed under the simplifications described in this section. For the theoretical discussions, we will continue referring to the general setting from previous sections.

To wrap up this section, in Fig. 6 we show the results of the QAMC for different payoffs depending on the underlying price at maturity when the modified iterative amplitude estimation (mIQAE) algorithm (see [20]) is used. The mIQAE is considered the current state of the art with regard to amplitude estimation algorithms.

For all experiments except for the cliquet option, we have encoded the Black-Scholes probability distribution with risk-free rate 0.01 and volatility 0.5. Moreover, we have considered a one-year maturity and an initial underlying value of 1.0. For the discretization of the distribution, we have considered 32 points between 0.01 and 5.0, thus requiring the use of 5 qubits.

For the cliquet option, we have encoded the Black-Scholes probability distribution with risk-free rate $r = 0.05$ and volatility $\sigma = 0.5$ in (19), starting from the initial underlying price $S_0 = 1.0$. Moreover, we have considered a single reset date of one year and a maturity of two years. For the discretization of the distribution we have considered 1024 paths consisting of 32 points for the reset date and 32 points for the maturity, both ranging between 0.01 and 5.0, thus requiring $5 \cdot 2 = 10$ qubits.

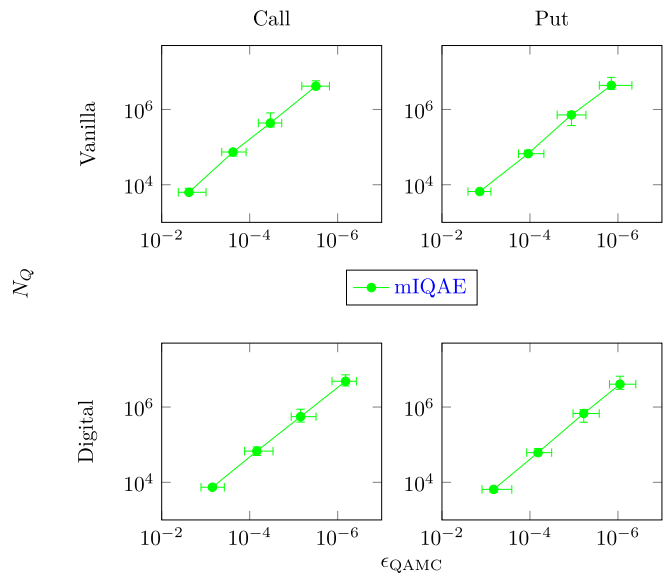


Figure 6 Absolute error between the QAMC algorithm and the discretized expectation versus the respective number of calls to the oracle for different precisions ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. Each of the panels corresponds to the payoff of different options. The experiments have been performed using the square root encoding

3 Alternative schedule for QAMC

As it is shown in Sect. 2.2.2, by sampling from the quantum circuit we can obtain an estimation $\widetilde{P}_{|0\rangle}$ of:

$$\frac{1}{\|\sqrt{F(S)}\|_\infty} \sum_{k=0}^{\mathcal{K}-1} |p_S(S_k)F(S_k)| \,. \tag{21}$$

Nevertheless, it is important to note that, for derivatives whose payoffs can become negative, the naive use of this method will not yield to correct prices approximations. In order to illustrate this, suppose that there is a payoff of the form (see Fig. 2):

$$F(S_T) = S_T - K, \tag{22}$$

with T being the maturity of the contract, S_T the price of the underlying at maturity and K the strike price of the contract (see [1] for details).

Figure 7 shows the results for the square root encoding combined with the mIQAE for a naive implementation of QAMC. It illustrates that there is no convergence to the correct value because of the presence of the absolute value.

In order to avoid the errors introduced by the presence of the absolute values in the QAMC, whenever we have a payoff that is potentially negative, we must divide our problem into two distinct problems. On the one hand, we must define the positive part of our target function:

$$F_+(S_T) = \max(F(S_T), 0). \tag{23}$$

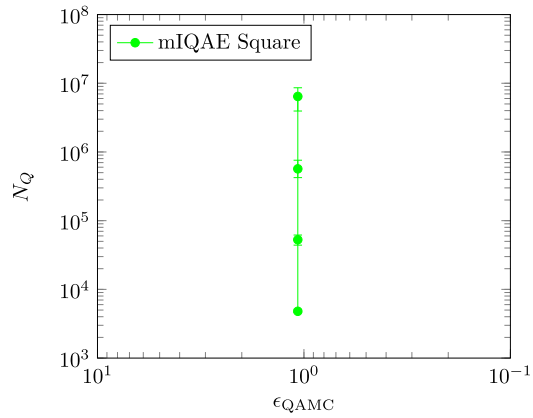


Figure 7 Absolute error between the QAMC algorithm and the discretized expectation of an option with a payoff $(S_T - K)$ with $K = 1.5S_t$ versus the number of calls to the oracle for different values of precision ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. The experiments have been performed using the square root encoding

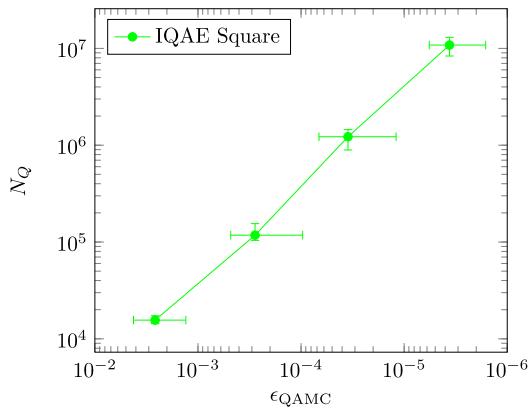


Figure 8 Absolute error between the QAMC algorithm and the discretized expectation of an option with a payoff $(S_T - K)$ with $K = 1.5S_t$ versus the number of calls to the oracle for different values of precision ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. The experiments have been performed using the square root encoding separating the positive and negative parts of the payoff

On the other hand, we define the negative part of our target function:

$$F_-(S_T) = |\min(F(S_T), 0)| \,. \tag{24}$$

Therefore, we can express our payoff as a linear combination of the positive and negative parts:

$$F(S_T) = F_+(S_T) - F_-(S_T). \tag{25}$$

In terms of estimation we need to perform a separate estimation of both the positive and negative parts and then combine the results. Applying this decomposition, the mIQAE provides the results in Fig. 8.

The same strategy applies when we have a path dependent payoff. The main difference is that our payoff function now depends on the whole path.

In Sects. 3.1 and 3.2 we develop a new strategy which does not require the user to separate the problem into two. On the one hand, a new encoding is proposed. On the other hand, a different amplitude estimation technique is employed.

3.1 Direct encoding

The *direct encoding* algorithm starts from the same initial state $|S\rangle$:

$$|S\rangle = U_S|0\rangle = \sum_{k=0}^{\mathcal{K}-1} \sqrt{p_S(S_k)}|S_k\rangle, \tag{26}$$

where $\mathcal{K}$ is again the number of possible paths defined by the given discretization. The next step is to define the operator U_F such that pushes the payoff without squared roots into the amplitude. For this purpose, an additional single qubit register is needed, so that:

$$\begin{aligned} |F\rangle = U_F|S\rangle|0\rangle = \frac{1}{\|F(S)\|_\infty} \sum_{k=0}^{\mathcal{K}-1} & \sqrt{p_S(S_k)}F(S_k)|S_k\rangle|0\rangle \\ & + \sqrt{p_S(S_k)}\,(1 - F(S_k))\,|S_k\rangle|1\rangle. \end{aligned} \tag{27}$$

Next, we apply the inverse of the U_S unitary on the state $|F\rangle$ (see Fig. 9), thus getting:

$$U_S^\dagger|F\rangle = \frac{1}{\|F(S)\|_\infty} \sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k)|0\rangle|0\rangle + \cdots. \tag{28}$$

We will denote by U_{DE} the application of the whole pipeline:

$$U_{\mathrm{DE}} := U_S^\dagger U_F U_S. \tag{29}$$

The square root probability of measuring the eigenstate zero is:

$$\sqrt{P_{\overline{|0\rangle}}} = |\langle 0|U_{\mathrm{DE}}|0\rangle| = \frac{1}{\|F(S)\|_\infty} \left| \sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k) \right|. \tag{30}$$

Note the difference between $P_{|0\rangle}$ from the square root encoding in Equation (16) and $P_{\overline{|0\rangle}}$ from the direct encoding in Equation (30). The former one refers to the probability of measuring zero in the last register when the unitary U_{SRE} is applied. The latter

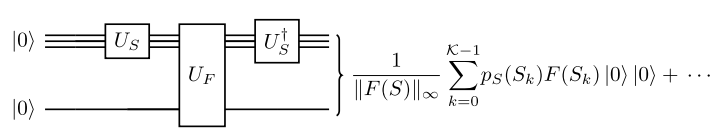


Figure 9 Scheme of the generation of the oracle in the direct encoding. The gate U_S corresponds to Equation (12). The gate U_F corresponds to Equation (27)

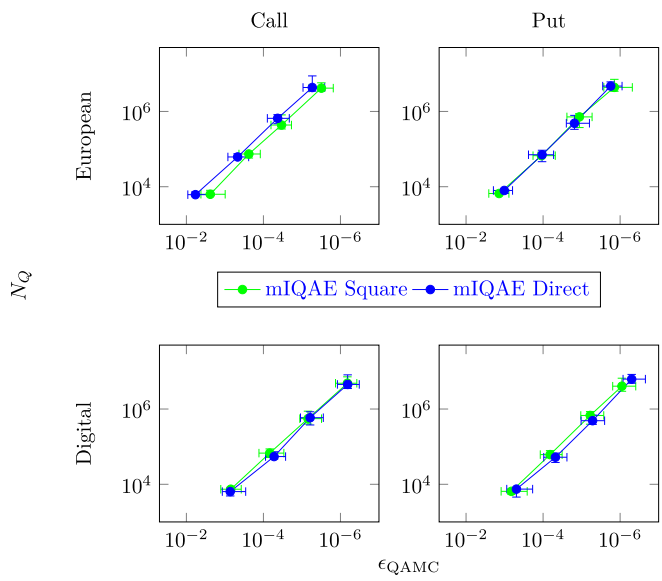


Figure 10 Absolute error between the QAMC algorithm and the discretized expectation versus the respective number of calls to the oracle for different precisions ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. Each of the panels corresponds to the payoff of a different option. The experiments have been performed using the square root and direct encodings

refers to the probability of measuring the eigenstate zero when the unitary U_{DE} is applied. Finally, we apply an amplitude estimation algorithm to state zero of the oracle U_{DE} to obtain an estimate $\widetilde{P}_{\overline{|0\rangle}}$ of $P_{\overline{|0\rangle}}$, thus getting an estimation of the expectation in Equation (3):

$$\begin{aligned}\mathbb{E}[F(S_T)|\mathcal{F}_t] &= \sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k) + \epsilon_{\mathcal{K}} + \epsilon_{\text{EM}} \\ &\approx \|F(S)\|_\infty \sqrt{\widetilde{P}_{\overline{|0\rangle}}} + \epsilon_{\mathcal{K}} + \epsilon_{\text{EM}} + \epsilon_{\text{QAMC}},\end{aligned} \tag{31}$$

where $\sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k)$ is the discretized expectation, ϵ_{EM} is same Euler-Maruyama error as in Equation (5), $\epsilon_{\mathcal{K}}$ is the discretization error, $\widetilde{P}_{|0\rangle}$ is an estimation of the probability of obtaining zero in the last register and ϵ_{QAMC} is the sampling error given by,

$$\epsilon_{\text{QAMC}} = \left| \sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k) - \|F(S)\|_\infty \sqrt{\widetilde{P}_{\overline{|0\rangle}}} \right|. \tag{32}$$

This procedure allows pricing options with negative payoffs when the expected value is positive. However, the presence of the outer absolute value in Equation (30) still prevents from a correct estimation when negative expectations arise.

In Fig. 10 we show the results obtained for the same payoffs as in Fig. 6 with both the direct and the square root encoding. As we can see from the figure, the impact of using one or the other encoding is minimal in practice.

3.2 Amplitude estimation: mRQAE

In the previous section it was discussed that the discretized expectation can be estimated through the probability of measuring the eigenstate zero of U_{DE}:

$$\sqrt{P_{|0\rangle}} = \frac{1}{\|F\|_\infty}\left|\sum_{k=0}^{\mathcal{K}-1} p_S(S_k)F(S_k)\right|.$$

Thus, this partially solves the initial problem. Instead of obtaining the sum of absolute values, something proportional to the absolute value of the sum is returned. Hence, in a situation where the sign of the expectation is of interest, an additional mechanism to overcome this issue is needed. In fact, this is usually the case in financial applications, where the sign makes the difference between a profit and a loss.

For this case, we introduce the modified real quantum amplitude estimation (mRQAE) algorithm. The mRQAE is a modified version of the real quantum amplitude estimation (RQAE) (see [9]). The main feature of both algorithms is that they are able to read out the size and the sign of the target amplitude. They internally work performing several iterations where, in each iteration they use the Grover amplification algorithm to incrementally amplify the probability of obtaining the target quantum state. The main difference between them is that the number of calls to the amplified state N_i, the confidence on each iteration γ_i and the required precision on each iteration ϵ_i^p are chosen in a different manner. In turn, this makes the mRQAE asymptotically more efficient. More precisely, in Table 2 we show the performance of the mRQAE and the RQAE measured in terms of the number of calls to the oracle N_Q for a given precision ϵ and confidence γ along with the performance of other popular amplitude estimation algorithms in the literature including the aforementioned mIQAE.

Table 2 Performance of different amplitude estimation algorithms. N_Q denotes the number of calls to the oracle, ϵ is the target precision and $1-\gamma$ is the confidence level. Other parameters appearing in the table are related to each specific algorithm. For a full description of their meaning, the reader is referred to the associated references. The $\mathcal{O}$ symbol indicates the scaling behaviour of the performance, while the < indicates that the performance is proved rigorously

Algorithm	Performance
Monte Carlo	$N_Q^{\mathrm{MC}} = \mathcal{O}\left(\frac{1}{\epsilon^2}\right)$
QPE [17]	$N_Q^{\mathrm{QPE}} = \mathcal{O}\left(\frac{1}{\epsilon}\right)$
MLAE-LIS [21]	$N_Q^{\mathrm{LIS}} = \mathcal{O}\left(\frac{1}{\epsilon^{\frac{4}{3}}}\right)$
MLAE-EIS [21]	$N_Q^{\mathrm{EIS}} = \mathcal{O}\left(\frac{1}{\epsilon}\right)$
PLAE [22]	$N_{\mathcal{A}}^{\mathrm{PLAE}} = \mathcal{O}\left(\frac{1}{\epsilon^{1+\beta}}\right), d = \mathcal{O}\left(\frac{1}{\epsilon^{1-\beta}}\right)$
Improved MLAE [23]	$N_Q^{\mathrm{impMLAE}} = \mathcal{O}\left(\frac{1}{\epsilon}\frac{1}{d}\log(\frac{1}{\gamma})\right), d = 2^{q-2}$
IQAE [24]	$N_Q^{\mathrm{IQAE}} < \frac{50}{\epsilon}\log\left(\frac{2}{\gamma}\log_2\frac{\pi}{4\epsilon}\right)$
mIQAE [20]	$N_Q^{\mathrm{mIQAE}} < \frac{123}{\epsilon}\log\frac{6}{\gamma}$
QCoin [25]	$N_Q^{\mathrm{QCoin}} = \mathcal{O}\left(\frac{1}{a}\frac{1}{\epsilon}\log\frac{1}{\gamma}\right), k \geq 2, 1 \geq q \geq (k-1)$
QoPrime [22]	$N_Q^{\mathrm{QoPrime}} < C\lceil\frac{k}{q}\rceil\frac{1}{\epsilon^{1+q/k}}\log\left(\frac{4}{\gamma}\lceil\frac{k}{q}\rceil\right), d = \mathcal{O}\left(\frac{1}{\epsilon^{1-q/k}}\right)$
FasterAE [26]	$N_Q^{\mathrm{fasterAE}} < \frac{4.1\cdot 10^3}{\epsilon}\log\left(\frac{4}{\gamma}\log_2\left(\frac{2\pi}{3\epsilon}\right)\right)$
AdaptiveAE [27]	$N_Q^{\mathrm{adaptiveAE}} < \mathcal{O}\left(\frac{1}{\epsilon}\log\left(\frac{\pi^2(T+1)}{3\gamma}\right)\right), T = \lceil\frac{\log\frac{\pi}{K\epsilon}}{\log K}\rceil$
RQAE [9]	$N_Q^{\mathrm{RQAE}} < \frac{C_1(q)}{\epsilon_a}\log\left[\frac{3.3}{\gamma}\log_q\left(\frac{C_2(q)}{\epsilon}\right)\right]$
mRQAE (70)	$N_Q^{\mathrm{MRQAE}} < \frac{C_1(q)}{\epsilon}\log\left[\frac{C_2(q)}{\gamma}\right]$

Algorithm 1 mRQAE pseudocode

Input:

ϵ // Required precision
γ // $1-\gamma$ is the confidence level
q // Amplification policy
$\mathcal{A}$ // Oracle

Output:

a // Estimated amplitude with sign

Algorithm:

// Define relevant parameters
Set $\epsilon^p(q,\infty) = \frac{1}{2}\sin^2\left(\frac{\pi}{4q}\right)$, $\epsilon^p(q,0) = \frac{1}{2}\sin\left(\frac{\pi}{2(q+2)}\right)$
Set $T = \log_q\left(q^2\frac{2\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)}\right)$, $k^{\max} = \left\lceil \frac{\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)} - \frac{1}{2}\right\rceil$
$i = 1$ // First Iteration
Set $\gamma_i = \frac{\gamma}{2}\frac{q-1}{q}\frac{1}{2k^{\max}+1}$ // Confidence level parameter
Set $N_i = \left\lceil \frac{1}{2(\epsilon^p(q,0))^2}\log\left(\frac{2}{\gamma_i}\right)\right\rceil$ // Number of shots
Set $\epsilon_i^p = \sqrt{\frac{1}{2N_1}\log\left(\frac{2}{\gamma_i}\right)}$
Set $b = 0.5$ // Shift
Build $\mathcal{A}_b$ from $\mathcal{A}$ and b (see Appendix B) and measure $\hat{p}_{\mathbf{sum}}$
Build $\mathcal{A}_{-b}$ from $\mathcal{A}$ and b (see Appendix B) and measure $\hat{p}_{\mathbf{diff}}$
$a^{\max} = \min\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b} + \frac{\epsilon_i^p}{|2b|}, 1\right)$, $a^{\min} = \max\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b} - \frac{\epsilon_i^p}{|2b|}, -1\right)$
$a = \frac{a^{\max}+a^{\min}}{2}$
$\epsilon^a = \frac{a^{\max}-a^{\min}}{2}$ // Following Iterations
while $\epsilon^a > \epsilon$ **do**
  $i = i+1$
  Set $b = -a^{\min}$ // Shift
  Set $k = \left\lfloor \frac{\pi}{4\arcsin(2\epsilon^a)} - \frac{1}{2}\right\rfloor$ // Number of amplifications
  $k = \min(k, k^{\max})$
  Set $\epsilon^p(q,k) = \frac{1}{2}\sin^2\left(\frac{\pi}{4\left(q+\frac{2}{2k+1}\right)}\right)$
  Set $\gamma_i = \frac{\gamma}{2}\frac{q-1}{q}\frac{2k+1}{2k^{\max}+1}$ // Confidence level parameter
  Set $N_i = \left\lceil \frac{1}{2(\epsilon^p(q,k))^2}\log\left(\frac{2}{\gamma_i}\right)\right\rceil$ // Number of shots
  Set $\epsilon_i^p = \sqrt{\frac{1}{2N_i}\log\left(\frac{2}{\gamma_i}\right)}$
  Build $\mathcal{A}_b$ from $\mathcal{A}$ and b (see Appendix B).
  Build the corresponding Grover operator from $\mathcal{A}_b$: $\mathcal{G}^k(\mathcal{A}_b)$ and measure p
  $p^{\max} = \min(p+\epsilon_i^p, 1)$, $p^{\min} = \max(p-\epsilon_i^p, 0)$
  $\theta^{\max} = \frac{\arcsin(\sqrt{p^{\max}})}{2k+1}$, $\theta^{\min} = \frac{\arcsin(\sqrt{p^{\min}})}{2k+1}$
  $a^{\max} = \sin(\theta^{\max}) - b$, $a^{\min} = \sin\left(\theta^{\min}\right) - b$
  $a = \frac{a^{\max}+a^{\min}}{2}$
  $\epsilon^a = \frac{a^{\max}-a^{\min}}{2}$
end while
return a

In Algorithm 1 there is a schematic description of the code for the mRQAE, where the bold statements show the parts of the algorithm executed on a quantum computer. For the mRQAE results presented in this article, the unitary operator $\mathcal{A}$ provided to the algorithm is U_{DE} (29). For more details on the properties of the method, we refer the reader to the Appendix B.

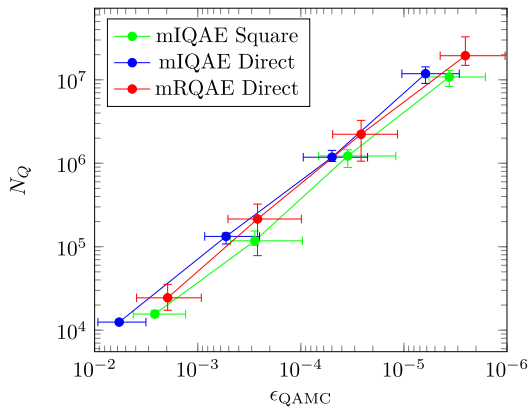


Figure 11 Absolute error between the QAMC algorithm and the discretized expectation of an option with a payoff $(S_T - K)$ with $K = 1.5S_t$ versus number of calls to the oracle for different values of precision ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. For the mRQAE we have used the direct encoding and for the mIQAE we have applied both techniques. Moreover, in the case of the mIQAE we have separated the negative and positive parts of the payoff for a correct pricing

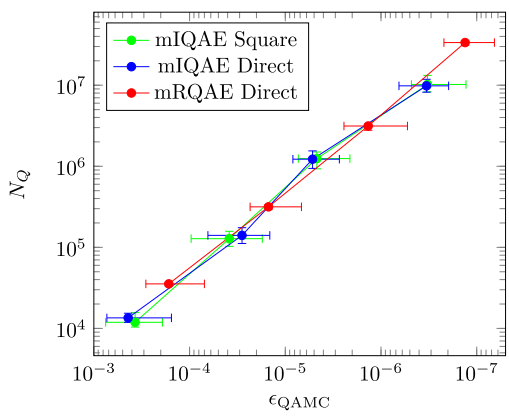


Figure 12 Absolute error between the QAMC algorithm and the discretized expectation of a cliquet option with $c = -0.1$, $f = 0.1$, $c_g = -0.2$ and $f_g = 0.2$, versus number of calls to the oracle for different values of precision ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. For the mRQAE we have used the direct encoding and for the mIQAE we have applied both techniques. Moreover, in the case of the mIQAE we have separated the negative and positive parts of the payoff for a correct pricing

In Fig. 11, an example where the price of the derivative becomes negative is shown. As it is shown, mRQAE is able to recover the true price without requiring additional mechanisms. Moreover, it is competitive with the mIQAE which is currently considered one of the most efficient algorithms in the literature in terms of number of calls to the oracle.

Figure 12, shows another example where the price of a cliquet option becomes negative. Once again, apart from not requiring any additional mechanisms, the figure shows that the new pipeline is competitive with the standard one when the mIQAE algorithm is considered.

In Fig. 13 we show that the mRQAE obtains a reasonable performance compared with the mIQAE when we use it for contracts with positive payoff.

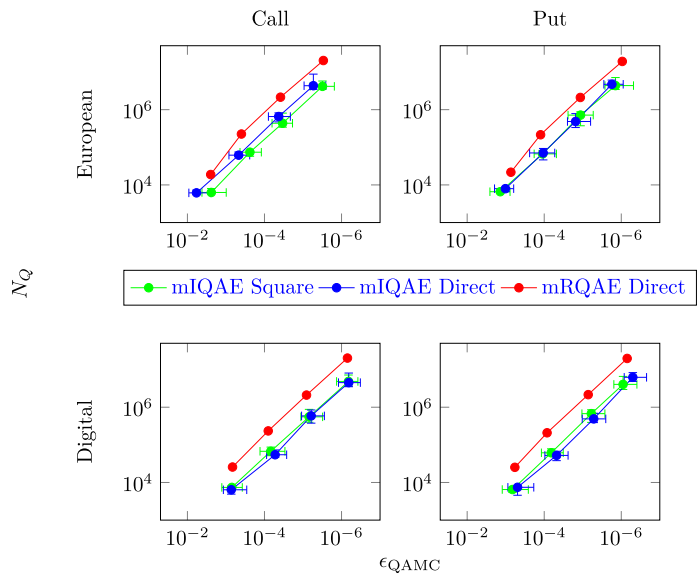


Figure 13 Absolute error between the QAMC algorithm and the discretized expectation versus the respective number of calls to the oracle for different precisions ϵ. The dots represent the medians and the error bars the 25 and 75 percentiles. Each of the panels corresponds the payoff of a different option. For the mRQAE we have used the direct encoding and for the mIQAE we have applied both techniques

Table 3 Hardware and software configuration for the experiments

Provider	CESGA	CESGA
Node	QLM QAPTIVA	ILK
CPU	48 Intel(R) Xeon(R) Platinum 8260L	2 Intel Xeon Ice Lake 8352Y
RAM	1510 GB	256 GB
OS	Red Hat Enterprise Linux release 9.2	Rocky Linux release 8.4
Python version	3.9.16	3.9.9
QLM version	QLM-1.9.1	myQLM-1.9.9

3.3 Computer implementation details

Concerning the hardware, all the tests have been performed the Finisterrae III (FT3) infraestructure provided by CESGA. On the software side, we have mainly used our *Python* library [28] which is built on top of the QLM library provided by Atos. In Table 3 we provide further details.

4 Conclusions

In this article, we have presented a novel proposal to leverage quantum computers for derivative pricing. This proposal combines the direct encoding protocol with the mRQAE algorithm to price derivative products with negative payoffs without the need of separating the payoff function into positive and negative codomains, as it is mandatory in the standard QAMC algorithm to obtain correct prices. Moreover, we have performed several experiments to validate our methodology and showed that the new methodology is competitive in terms of "speed" with the current standards.

The direct encoding introduces an alternative way to perform QAMC. It is somehow more natural than its predecessors, since we directly work with the payoffs and not with

their square roots. However, in order to obtain the most of it, we need to combine it with algorithms such as the mRQAE. The mRQAE is capable of reading out the sign of the underlying amplitude and offers some control over the depth of the circuits, a crucial feature in the current NISQ era.

Although in theory QAMC exhibits a quadratic speedup over CMC, there are still many issues to solve in practice, specially if we take into consideration the current state-of-the-art hardware constraints. First, as explained in Sect. 2.2.1, the implementation of the oracle U_S requires an excessively large number of qubits. Second, the depths required by the current Grover-like routines are not feasible under the current decoherence times. Finally, the total number of gates when combining the implementation of oracle U_S with a Grover-like algorithm requires a gate error beyond the capabilities of the current technology.

The possible extensions or improvements of the proposed techniques are all centered around addressing the previously pointed problems. Here we mention three areas that might be of broad interest for the QAMC community.

First, although we have seen that it is theoretically possible to efficiently initialize a state through the direct simulation of a target SDE, it is too demanding in terms of hardware requirements. For this reason, we need to consider new ways to efficiently initialize states that approximately encode the target probability distribution. This is the main step towards a real end to end implementation of the QAMC.

A second research area aims to investigate the proposal of new encodings. Both the direct and the square encodings suffer from the fact that the payoff needs to be normalized. Otherwise, we need to define an upper bound for the payoff, which induces a truncation error. Other encodings could work with different normalizations, potentially reducing the effects of truncation.

Third, we point towards possible extensions of the mRQAE. The choice of parameters considered in the present paper is not unique and alternative choices could be more efficient in terms, for example, of the total number of shots. In fact, we believe that it is interesting to explore different parameter settings in the future.

Appendix A: Some useful inequalities

In Appendix A we prove several lemmas that state some inequalities that are used in this article.

Lemma 1 *For all $a, b \in [0,1]$ such that $a > b$, we have that*:

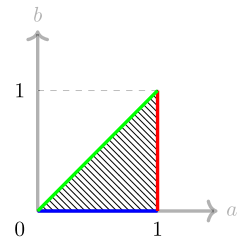
$$\sin(a) - \sin(b) \leq \sin(a - b).$$

Proof In order to prove it, we will show that the function:

$$f(a,b) = \sin(a) - \sin(b) - \sin(a - b), \tag{33}$$

is less or equal than zero for the region delimited by

$$
\begin{aligned}
&b = 0,\ a \in [0,1] \text{ marked in blue,}\\
&a = b,\ a,b \in [0,1] \text{ marked in green,}\\
&a = 1,\ b \in [0,1] \text{ marked in red.}
\end{aligned}
\tag{34}
$$



First we show that $f \leq 0$ on the boundaries:

- In the blue region $b = 0$ we have that $f = 0$.
- In the green region $a = b$ we have that $f = 0$.
- In the red region we have that $f(1,b) = \sin(1) - \sin(b) - \sin(1-b)$. In this region it is not obvious that the function is smaller than zero. However, it is easy to verify that $f(1,b)$ has a global minimum at $b = 0.5$ which is smaller than zero. In the region $b \in [0, 1/2)$ is monotonically decreasing and in the region $b \in (1/2, 1)$ is monotonically increasing with $f(1,1) = 0$, so that $f(1,b) \leq 0$.

In the interior it is enough to notice that $\frac{\partial f}{\partial a} \leq 0\, \forall a \in (0,1)$. If we start in the green boundary where the function is equal to zero and we move along the a axis, the function strictly decreases. Therefore, it must be smaller of equal than zero.

As we have proven that $f \leq 0$ in the interior plus the boundaries we have proven the inequality. □

Lemma 2 *For all $a, b \in [0,1]$ $a > b$, we have that*:

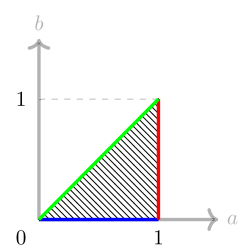
$$
\sin(a) - \sin(b) \geq \sin\left(\frac{a-b}{2}\right).
$$

Proof In order to prove it, we will show that the function:

$$
f(a,b) = \sin(a) - \sin(b) - \sin\left(\frac{a-b}{2}\right), \tag{35}
$$

is greater than zero for the region delimited by

$$
\begin{aligned}
&b = 0,\ a \in [0,1] \text{ marked in blue,}\\
&a = b,\ a,b \in [0,1] \text{ marked in green,}\\
&a = 1,\ b \in [0,1] \text{ marked in red.}
\end{aligned}
\tag{36}
$$



First we use a trigonometric identity on the sum of sine functions and transform Equation (35) into:

$$\begin{aligned} f(a,b) &= 2\cos\left(\frac{a+b}{2}\right)\sin\left(\frac{a-b}{2}\right) - \sin\left(\frac{a-b}{2}\right) \\ &= \left(2\cos\left(\frac{a+b}{2}\right) - 1\right)\sin\left(\frac{a-b}{2}\right). \end{aligned} \tag{37}$$

Next, we show that the two factors in the product are positive. For the first term:

$$\sin\left(\frac{a-b}{2}\right), \tag{38}$$

this is straightforward. The second term:

$$\left(2\cos\left(\frac{a+b}{2}\right) - 1\right)$$

has a global minimum $2\cos(1) - 1$ at $a = b = 1$.

Therefore, since both factors are positive in the prescribed region, the product is positive. □

Lemma 3 *For all $a, b \in [0, \arcsin(1)]$ such that $a > b$, we have that*:

$$\arcsin(a) - \arcsin(b) \leq 2\arcsin(a-b).$$

Proof From Lemma 2 we know that for $\alpha, \beta \in [0,1]$ we have that:

$$\sin(\alpha) - \sin(\beta) \geq \sin\left(\frac{\alpha-\beta}{2}\right).$$

Now we define the two variables $a = \arcsin(\alpha)$ and $b = \arcsin(\beta)$ and substitute them into the previous equation getting:

$$a - b \geq \sin\left(\frac{\arcsin a - \arcsin b}{2}\right).$$

Taking the arcsin on both sides we get the desired identity. Note that this can be done since the arcsin in the defined interval is a positive increasing function. □

Lemma 4 *For all $a, b \in [0,1]$ such that $a > b$, we have that*:

$$\sqrt{a} - \sqrt{b} \leq \sqrt{a-b}.$$

Proof If we assume that the stated inequality does not hold, by taking the square on both sides of the resulting expression, we get:

$$a + b - 2\sqrt{ab} > a - b \implies b > \sqrt{ab} \implies \sqrt{b} > \sqrt{a} \implies b > a,$$

which contradicts the hypothesis $a > b$. □

The following lemma is a modification of Lemma 3.4 in [20]:

Lemma 5 *Consider an increasing sequence* $x_0 = 1, x_1, \ldots x_N$ *such that* $x_N \leq x_{\max}$ *and* $x_{i+1} \geq q x_i \; \forall i \in \{0, \ldots, N-1\}$, $q > 1$. *Moreover, suppose that* f *is a positive increasing function over the range* $[1, x_{\max}]$, *then*:

$$\sum_{i=1}^{N} f(x_i) \leq \sum_{i=0}^{N-1} f\left(\frac{x_{\max}}{q^i}\right). \tag{39}$$

Proof First note that:

$$x_j \leq \frac{x_N}{q^{N-j}} \leq \frac{x_{\max}}{q^{N-j}}.$$

Since f is a positive increasing function,

$$f(x_j) \leq f\left(\frac{x_{\max}}{q^{N-j}}\right) \implies \sum_{j=1}^{N} f(x_j) \leq \sum_{j=1}^{N} f\left(\frac{x_{\max}}{q^{N-j}}\right) = \sum_{j=0}^{N-1} f\left(\frac{x_{\max}}{q^{N-j}}\right). \qquad \square$$

Appendix B: Modified real quantum amplitude estimation

Consider a one-parameter family of oracles $\mathcal{A}_b$ that, acting on the state $|0\rangle$, yield

$$\mathcal{A}_b|0\rangle = |\psi\rangle = (a+b)\,|\phi\rangle + c_b\,|\phi^{\perp}\rangle_b\,, \tag{40}$$

where a is a real number, b is an auxiliary, continuous and real parameter that we call "shift", and $|\phi\rangle$ is a specified direction in the Hilbert space $\mathcal{H}$. The mRQAE algorithm estimates the amplitude a exploiting the possibility of tuning the shift b iteratively. The ket $|\psi\rangle$ belongs to the plane $\Pi_b = \text{span}\{|\phi\rangle, |\phi^{\perp}\rangle_b\} \subset \mathcal{H}$ for which the kets $|\phi\rangle$ and $|\phi^{\perp}\rangle_b$ provide an orthonormal basis. Note that all the quantities with a sub-index b depend on the actual value of the shift. In practice, the construction of oracles such as $\mathcal{A}_b$ from a given un-shifted oracle $\mathcal{A}$ is generally not difficult. In most cases, a controlled shift of an amplitude can be efficiently implemented via Hadamard gates and some controlled operations. In particular, its implementation is straightforward in the framework described in [29].

Given a precision level ϵ and a confidence level $1-\gamma$, the goal of the algorithm is to compute a confidence interval $(a_I^{\min}, a_I^{\max}) \subset [-1, 1]$ of width smaller than 2ϵ which contains the value of a with probability greater or equal to $1-\gamma$ (see Fig. 14). Here, I denotes the number of iterations to achieve the prescribed accuracy. We take as a representative of the interval its center, $a_I = \frac{a_I^{\min} + a_I^{\max}}{2}$, thus admitting a maximum error of ϵ:

$$\mathbb{P}\Big[|a - a_I| \geq \epsilon\Big] \leq \gamma\,. \tag{41}$$

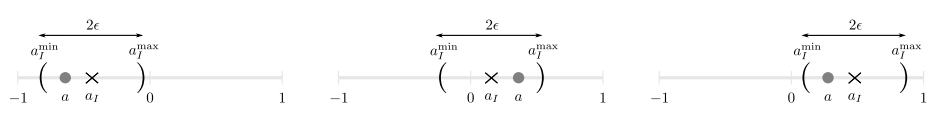


Figure 14 Illustration of the three positions of the confidence interval with respect to zero. The different symbols include a which is the target amplitude to be estimated, 2ϵ which is the width of the confidence interval with bounds $(a_I^{\min}, a_I^{\max})$ and a_I which is the center of the confidence interval

It is convenient to express the amplitudes in terms of their corresponding angles by means of the generic mapping $\theta_x = \arcsin(x)$ for any real amplitude x. Note that the angle representation is particularly suited to describe Grover amplifications, which indeed admit an interpretation as rotations in the plane Π_b. As an example, the state $|\psi\rangle$ given in (40) can be written as

$$|\psi\rangle = \sin(\theta_{a+b})|\phi\rangle + \cos(\theta_{a+b})|\phi^{\perp}\rangle_b \,, \tag{42}$$

where θ_{a+b} represents a rotation in the plane Π_b defined above. Throughout this chapter, we will be changing back and forth from the representation in terms of the actual amplitude or its associated angle whenever needed. In order to avoid notational clutter, we henceforth drop the sub-index b on the perpendicular ket, leaving its dependence on the shift as understood. Actually, such dependence does not play any role for the algorithm.

In the following subsections we describe step by step the inner workings of the mRQAE.

B.1 First iteration: estimating the sign

This step achieves a first estimation of the bounds of the confidence interval $(a_1^{\min}, a_1^{\max})$. Normally, this estimation would not be sensitive to the sign of the underlying amplitude because, when sampling from a quantum state, we obtain the square of the amplitude. Nevertheless, by taking advantage of the shift b we can circumvent this limitation. In order to compute the sign, we will combine two different pieces of information: the result of measuring the two oppositely shifted states $|\psi_1\rangle_\pm$ defined as:

$$|\psi_1\rangle_- := (a - b_1)\,|\phi\rangle + \cdots \qquad |\psi_1\rangle_+ := (a + b_1)\,|\phi\rangle + \cdots \,,, \tag{43}$$

for an arbitrary real shift b_1. The sign of b_1 has to be decided at the start of the algorithm to have a clear reference. In practice, in some setups it is possible to measure at the same time both states taking advantages of Hadamard gates as in the quantum arithmetic techniques discussed in [29]. As a and b_1 are real numbers, we have the identity:

$$a = \frac{(a + b_1)^2 - (a - b_1)^2}{4b_1} \,, \tag{44}$$

and we can build a first empirical estimation $\hat{a}_1$ of a as follows:

$$\hat{a}_1 = \frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} \,, \tag{45}$$

where $\hat{p}_{\text{sum}}$ and $\hat{p}_{\text{diff}}$ are the empirical probabilities of getting $|\phi\rangle$ when measuring $|\psi_1\rangle_-$ and $|\psi_1\rangle_+$, respectively. Throughout this chapter, when we measure, we will use $\hat{p}$ to denote the empirical probability obtained from direct sampling. As an example, if in iteration i we sample the state N_i times, getting $|\phi\rangle$ as an outcome $\hat{N}_i$ times, the estimated probability of $|\phi\rangle$ will be $\hat{p}_i = \frac{\hat{N}_i}{N_i}$.

From (44) and (45), we can obtain a first confidence interval $(a_1^{\min}, a_1^{\max})$, with:

$$a_1^{\min} = \max\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} - \frac{\epsilon_1^p}{|2b_1|}, -1\right), \quad a_1^{\max} = \min\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} + \frac{\epsilon_1^p}{|2b_1|}, 1\right), \tag{46}$$

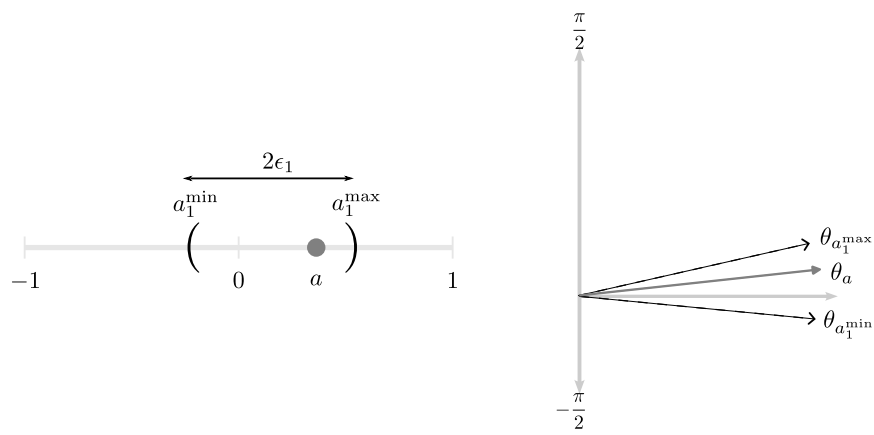


Figure 15 First confidence interval in terms of amplitudes an angles. In the figure on the left, the dot corresponds to a, namely the probability to be estimated; $a_1^{\min}$ and $a_1^{\max}$ define the confidence interval whose width is $2\epsilon_1$. In the figure on the right, the same confidence interval is represented in terms of angles. Note that the "true value" (represented by either a and θ_a) falls inside the confidence interval. In order to avoid clutter, here we are not representing the central value of the confidence interval

so that,

$$a_1 = \frac{a_1^{\max} + a_1^{\min}}{2}\,, \quad \epsilon_1^a = \frac{a_1^{\max} - a_1^{\min}}{2}\,, \tag{47}$$

where the max and the min operations are introduced because we know a priori that probabilities are bounded between 0 and 1. The assignment of an error ϵ_1^p to the empirical result $\hat{p}_1$ relies on a statistical analysis and depends on the employed statistical bound, such as Chebysev, Chernoff (Hoeffding) and Clopper-Pearson bounds. Here one of the main differences with respect to other algorithms in the literature becomes obvious: although the probabilities are bounded between 0 and 1, the estimated amplitude obtained by the identity (45) is now bounded between $-1 \leq a_1 \leq 1$. In other words, a_1 *can be negative* (see Fig. 15). Note that the sign of the amplitude depends on the sign of b_1, which is taken positive for simplicity. However, this election is arbitrary and it could be chosen negative.

B.2 Following iterations: amplifying the probability and shrinking the interval

On consecutive iterations, given an input confidence interval $(a_i^{\min}, a_i^{\max})$ (see Fig. 16a) we want to obtain a tighter one $(a_{i+1}^{\max}, a_{i+1}^{\min})$ and iterate the process until the desired precision ϵ is reached. At each iteration, the process for narrowing the interval starts by choosing a new shift according to

$$b_{i+1} = -a_i^{\min}\,. \tag{48}$$

Note that, this election is not unique, as we could have chosen $b_{i+1} = -a_i^{\max}$ instead. Always keep in mind that the phase that we are obtaining is relative to the original value of $b = b_1$. By considering the choice (48), we force our lower bound to match exactly zero (see Fig. 16b). The boundaries of the confidence interval $(a_i^{\min}, a_i^{\max})$, when shifted and then expressed in terms of the corresponding angles, become:

$$\alpha_i^{\min} = 0, \quad \alpha_i^{\max} = \arcsin\left(a_i^{\max} - a_i^{\min}\right) = \arcsin\left(2\epsilon_i^a\right). \tag{49}$$

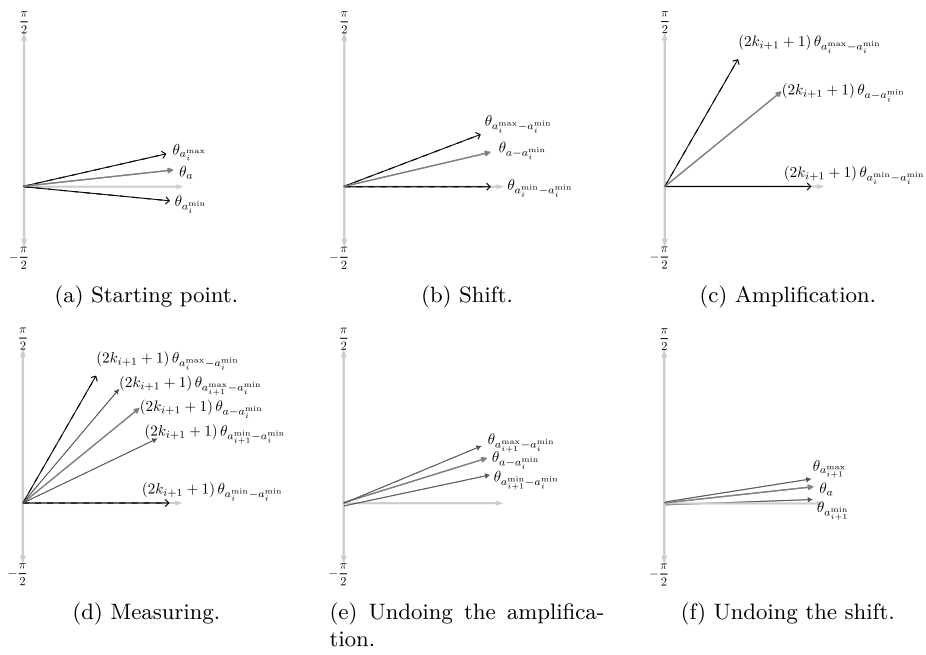


Figure 16 Graphical representation of the steps performed in the $i+1$ iteration. The solid grey line represents the unknown target value (which is not necessarily the center of the confidence interval). The dashed lines represent the bounds obtained at the i-th step, while the dotted lines represent the bounds obtained at step $i+1$

The angular region $\alpha_i^{\min} \leq \alpha_i \leq \alpha_i^{\max}$ represents the confidence interval and we refer to it as the confidence fan.

The next step takes advantage of the Grover operator, defined as

$$\mathcal{G} = -\mathcal{A}_b \mathcal{R}_{|0\rangle} \mathcal{A}_b^\dagger \mathcal{R}_{|\phi\rangle} \,, \tag{50}$$

where

$$\mathcal{R}_{|0\rangle} \quad = \mathbb{1} - 2|0\rangle\langle 0|, \quad \mathcal{R}_{|\phi\rangle} \quad = \mathbb{1} - 2|\phi\rangle\langle\phi| \,, \tag{51}$$

and $\mathcal{A}_b$ is the oracle defined in (40). The Grover operator applied k_{i+1} times transforms the generic angle θ into $(2k_{i+1}+1)\theta$, see Fig. 16c. Hence, the state $|\psi_{i+1}\rangle_+$ is transformed to:

$$\begin{aligned} |\psi_{i+1}\rangle_+ &= (a + b_{i+1})\,|\phi\rangle + c_i|\phi^\perp\rangle = \left(a - a_i^{\min}\right)|\phi\rangle + c_i|\phi^\perp\rangle \\ &\equiv \sin(\theta_{i+1})|\phi\rangle + \cos(\theta_{i+1})|\phi^\perp\rangle \\ &\xrightarrow{\mathcal{G}^{k_{i+1}}} \\ &\sin\left((2k_{i+1}+1)\theta_{i+1}\right)|\phi\rangle + \cos\left((2k_{i+1}+1)\theta_{i+1}\right)|\phi^\perp\rangle \,, \end{aligned} \tag{52}$$

where the sub-index "+" is employed as in (43), $\mathcal{G}^{k_{i+1}}$ indicates the Grover operator applied k_{i+1} times and, in the second equality, we use (48). Moreover, in (52) we have implicitly defined the angle $\theta_{i+1} \equiv \arcsin\left(a - a_i^{\min}\right)$ for which we will use the analogous notation $\theta_{a-a_i^{\min}} = \theta_{i+1}$, according to convenience of presentation.

In order to avoid ambiguities due to the lack of a bijective correspondence angle/amplitude, when measuring amplified probabilities, we cannot allow the amplified angles to go beyond $\left[0, \frac{\pi}{2}\right]$. Namely, we need the amplified confidence fan to stay within the first quadrant. Relying on (49), we choose the Grover amplification exponent as:

$$k_{i+1} = \left\lfloor \frac{\pi}{4 \arcsin\left(2\epsilon_i^a\right)} - \frac{1}{2} \right\rfloor , \tag{53}$$

so that we maximize the amplification factor while respecting the angle constraint.

Next, we measure the state $|\psi_{i+1}\rangle$ in the amplified space, obtaining the empirical probability

$$\hat{p}_{i+1} \approx \sin^2\left((2k_{i+1}+1)\theta_{i+1}\right) , \tag{54}$$

with the statistical error ϵ_{i+1}^p, and define:

$$p_{i+1}^{\min} := \max\left(\hat{p}_{i+1} - \epsilon_{i+1}^p, 0\right) , \quad p_{i+1}^{\max} := \min\left(\hat{p}_{i+1} + \epsilon_{i+1}^p, 1\right) , \tag{55}$$

and,

$$p_{i+1} := \frac{p_{i+1}^{\max} + p_{i+1}^{\min}}{2} , \tag{56}$$

where the max and min functions play an analogous role as in Sect. B.1 (see Fig. 16d).

In the next step we transform the angles corresponding to $p_{i+1}^{\max}$ and $p_{i+1}^{\min}$ to the non-amplified space:

$$\theta_{i+1}^{\min} = \frac{\arcsin\left(\sqrt{p_{i+1}^{\min}}\right)}{2k_{i+1}+1}, \quad \theta_{i+1}^{\max} = \frac{\arcsin\left(\sqrt{p_{i+1}^{\max}}\right)}{2k_{i+1}+1} . \tag{57}$$

In other words, we have just "undone" the amplification (see Fig. 16e).

Finally, we have to undo the shift (48), actually performing an opposite shift (see Fig. 16f). Using definitions analogous to those given in (46), we finally obtain:

$$a_{i+1}^{\min} = \sin\left(\frac{\arcsin\left(\sqrt{p_{i+1}^{\min}}\right)}{2k_{i+1}+1}\right) - b_{i+1} , \quad a_{i+1}^{\max} = \sin\left(\frac{\arcsin\left(\sqrt{p_{i+1}^{\max}}\right)}{2k_{i+1}+1}\right) - b_{i+1} , \tag{58}$$

so that,

$$\begin{aligned} a_{i+1} &= \frac{a_{i+1}^{\max} + a_{i+1}^{\min}}{2} , \\ \epsilon_{i+1}^a &= \frac{a_{i+1}^{\max} - a_{i+1}^{\min}}{2} = \frac{1}{2}\sin\left(\frac{\arcsin\left(\sqrt{p_{i+1}^{\max}}\right)}{2k_{i+1}+1}\right) - \frac{1}{2}\sin\left(\frac{\arcsin\left(\sqrt{p_{i+1}^{\min}}\right)}{2k_{i+1}+1}\right) . \end{aligned} \tag{59}$$

Recall that the goal is to reduce the width of the confidence interval until the desired precision ϵ is reached. For this purpose, one has to repeat the iteration just described until the goal is met.

Remark B.1 Throughout this section we have addressed the general structure of the algorithm. Nevertheless, we have not specified the values of all the involved parameters. More specifically, we have not discussed how ϵ_i^p is obtained. This parameter strictly depends on the number of shots on each iteration, N_i, and the confidence required on each iteration, $1-\gamma_i$, through a set of bounds such as Hoeffding's inequality or Clopper-Pearson bound (see [30, 31]). In Sect. B.3, more insight about these choices is provided.

B.3 Configuration and properties

As mentioned in Remark B.1, in order to complete the mRQAE method, we need to incorporate a particular choice for the parameters ϵ_i^p and b_1 and, thus, the parameters involved in its computation, *i.e.*, N_i and $1-\gamma_i$, the number of shots and confidence level of the i-th iteration, respectively. In the following theorem we describe a possible configuration, where there are three main parameters, ϵ, $1-\gamma$ and q. The first one is the target precision, the second one the target confidence level and the third one is the amplification policy. The amplification policy is a lower bound on the ratio between the number of aplications of the Grover oracle between subsequent iterations.

Theorem B.1 *Given ϵ, γ and q, and taking the parameters*:

$$N_i(q,\epsilon,\gamma) = \left\lceil \frac{1}{2\epsilon^p(q,k_i)^2} \log\left(\frac{2}{\gamma_i}\right) \right\rceil, \tag{60}$$

$$\gamma_i(q,\epsilon,\gamma) = \frac{\gamma}{2}\frac{q-1}{q}\frac{2k_i+1}{2k^{\max}+1}, \tag{61}$$

$$b_1 = \frac{1}{2}, \tag{62}$$

with

$$\epsilon^p(q,k_i) = \begin{cases} \frac{1}{2}\sin\left(\frac{\pi}{2(q+2)}\right), & \text{if} \quad k_i = 0 \\ \frac{1}{2}\sin^2\left(\frac{\pi}{4\left(q+\frac{2}{2k_i+1}\right)}\right), & \text{if} \quad k_i > 0 \end{cases}, \tag{63}$$

$$k^{\max}(q,\epsilon) = \left\lceil \frac{\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)} - \frac{1}{2} \right\rceil, \tag{64}$$

then, we have the following properties:

i) *The error ϵ_i^p at iteration i is bounded by*:

$$\epsilon_i^p \leq \epsilon^p(q,k_i)\,. \tag{65}$$

ii) *The amplification policy q_i is lower bounded at any iteration by*:

$$q_i = \frac{2k_{i+1}+1}{2k_i+1} \geq q\,. \tag{66}$$

iii) *The maximum depth of the circuit k_I is upper bounded by*:

$$k_I \leq k^{\max}(q, \epsilon). \tag{67}$$

iv) *The maximum number of iterations I is upper bounded by*:

$$I < T = \log_q \left(q^2 \frac{2 \arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)} \right) > I. \tag{68}$$

v) *The algorithm obtains a precision ϵ with confidence $1-\gamma$ (Proof of Correctness)*:

$$\mathbb{P}\Big[a \notin (a_I^{\min}, a_I^{\max}) \Big] \leq \gamma. \tag{69}$$

vi) *The total number of calls to the oracle is bounded by*:

$$N_Q < C_1(q) \frac{1}{\epsilon} \log\left(\frac{C_2(q)}{\gamma} \right), \tag{70}$$

where $C_1(q)$, $C_2(q)$ are two constants that depend on q.

Proof

Proof of property i) When finding an empirical estimate $\hat{p}$ of a probability p, we can assign to it a confidence interval (*i.e.* we can estimate an associated statistical error) by using Hoeffding's inequality[1] (see [30]):

$$\mathbb{P}\Big(\left| p - \hat{p} \right| \geq \epsilon_p \Big) \leq 2e^{-n\epsilon_p^2} = \gamma, \tag{71}$$

where ϵ_p is the precision, $1-\gamma$ is the confidence level and n is the number of shots (*i.e.* samplings) used for the measurement. As we fixed the values for N_i and γ_i in Equations (60) and (61), using (71) we get a fixed value for ϵ_i^p:

$$\mathbb{P}\Big[\left| \sin^2\left[(2k_i+1)\theta_i\right] - \hat{p}_i \right| \geq \epsilon_i^p \Big] \leq 2e^{-2N_i(\epsilon_i^p)^2} = \gamma_i. \tag{72}$$

Rewriting the previous expression in terms of ϵ_i^p we have:

$$\begin{aligned} \left(\epsilon_i^p\right)^2 &= \frac{1}{2N_i} \log\left(\frac{2}{\gamma_i}\right) = \\ &= \frac{1}{2\left[\frac{1}{2\epsilon^p(q,k_i)^2} \log\left(\frac{2}{\gamma_i}\right)\right]} \log\left(\frac{2}{\gamma_i}\right) \\ &\leq \frac{1}{2\left[\frac{1}{2\epsilon^p(q,k_i)^2} \log\left(\frac{2}{\gamma_i}\right)\right]} \log\left(\frac{2}{\gamma_i}\right) = (\epsilon^p(q,k_i))^2, \end{aligned} \tag{73}$$

where we used (60). We have thus proven property *i)*.

[1] Although there exist tighter bounds, they are much less tractable from an analytic point of view. One such example is Clopper-Pearson [31].

Proof of property ii) We first consider the case $i > 1$. From Equation (53), we have:

$$q_i = \frac{2k_{i+1}+1}{2k_i+1} = \frac{2\left\lfloor \frac{\pi}{4\arcsin(2\epsilon_i^a)} - \frac{1}{2} \right\rfloor + 1}{2k_i+1} . \tag{74}$$

We now consider the fact that $\lfloor x \rfloor \geq x - 1$ thus obtaining:

$$q_i \geq \frac{\frac{\pi}{2\arcsin(2\epsilon_i^a)} - 2}{2k_i+1} = \frac{\pi}{2\arcsin(2\epsilon_i^a)(2k_i+1)} - \frac{2}{2k_i+1} . \tag{75}$$

Now we focus on the term $(2k_i+1)\arcsin(2\epsilon_i^a)$ which can be rewritten in terms of ϵ_i^p as

$$\begin{aligned}(2k_i+1)\arcsin\Bigg[&\sin\left(\frac{\arcsin\left(\sqrt{\min(\hat{p}_i+\epsilon_i^p,1)}\right)}{2k_i+1}\right)\\ &-\sin\left(\frac{\arcsin\left(\sqrt{\max(\hat{p}_i-\epsilon_i^p,0)}\right)}{2k_i+1}\right)\Bigg] ,\end{aligned} \tag{76}$$

where we have used (58) and (55) after an obvious relabelling of the index.

In what follows, we use some lemmas in Appendix A. Using Lemma 1 we upper bound the expression (note that the values within the *sin* functions range between 0 and 1):

$$(2k_i+1)\arcsin(2\epsilon_i^a) \leq \arcsin\left(\sqrt{\min(\hat{p}_i+\epsilon_i^p,1)}\right) - \arcsin\left(\sqrt{\max(\hat{p}_i-\epsilon_i^p,0)}\right) . \tag{77}$$

Next, we upper bound the difference of arcsin functions using Lemma 3 with:

$$(2k_i+1)\arcsin(2\epsilon_i^a) \leq 2\arcsin\left(\sqrt{\min(\hat{p}_i+\epsilon_i^p,1)} - \sqrt{\max(\hat{p}_i-\epsilon_i^p,0)}\right) . \tag{78}$$

Then, using Lemma 4 we upper bound the difference of the square roots and get:

$$(2k_i+1)\arcsin(2\epsilon_i^a) \leq 2\arcsin\left(\sqrt{\min(\hat{p}_i+\epsilon_i^p,1) - \max(\hat{p}_i-\epsilon_i^p,0)}\right) . \tag{79}$$

After that, we simply upper bound the max and min functions by their values:

$$(2k_i+1)\arcsin(2\epsilon_i^a) \leq 2\arcsin\left(\sqrt{2\epsilon_i^p}\right) . \tag{80}$$

Finally, we can use Equation (80) to bound the expression in Equation (75), *i.e.*,

$$q_i \geq \frac{\pi}{2\arcsin(2\epsilon_i^a)(2k_i+1)} - 2 \geq \frac{\pi}{4\arcsin\left(\sqrt{2\epsilon_i^p}\right)} - \frac{2}{2k_i+1} . \tag{81}$$

As $\epsilon_i^p \leq \epsilon^p(q, k_i)$, we have that:

$$q_i \geq q \,. \tag{82}$$

So far, we have not treated the first iteration, $i = 1$, which we now consider explicitly:

$$q_1 = \frac{2k_2 + 1}{2k_1 + 1} = 2k_2 + 1 = 2\left\lfloor \frac{\pi}{4\arcsin(2\epsilon_1^a)} - \frac{1}{2} \right\rfloor + 1 \geq \frac{\pi}{2\arcsin(2\epsilon_1^a)} - 2 \,, \tag{83}$$

where we have recalled that $k_1 = 0$. We focus our attention on the term

$$\begin{aligned} \arcsin(2\epsilon_1^a) = \arcsin\Bigg[&\min\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} + \frac{\epsilon_1^p}{|2b_1|}, 1\right) \\ &- \max\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} - \frac{\epsilon_1^p}{|2b_1|}, -1\right)\Bigg] \,, \end{aligned} \tag{84}$$

where we have considered (46). Following the same strategy as before we define:

$$f(\epsilon_1^p) = \arcsin\left[\min\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} + \frac{\epsilon_1^p}{|2b_1|}, 1\right) - \max\left(\frac{\hat{p}_{\text{sum}} - \hat{p}_{\text{diff}}}{4b_1} - \frac{\epsilon_1^p}{|2b_1|}, -1\right)\right] \,. \tag{85}$$

An upper bound for (85) is:

$$\overline{f}(\epsilon_1^p) = \arcsin\left(\frac{\epsilon_1^p}{|b_1|}\right) \geq f(\epsilon_1^p) \,, \tag{86}$$

which can be obtained by using Lemma 3. Hence, from Equation (83), we have that:

$$q_1 \geq \frac{\pi}{2\arcsin\left(\frac{\epsilon^p}{|b_1|}\right)} - 2 \,. \tag{87}$$

Eventually, by the definition of b_1 we have:

$$q_1 \geq q \,, \tag{88}$$

and we have proven proposition *ii)*.

Proof of property iii) Using (58), if we get the maximum k, $k_{\max}$ we have

$$\epsilon_I^a(k^{\max}) = \frac{1}{2}\sin\left(\frac{\arcsin\left(\sqrt{p_I^{\max}}\right)}{2k^{\max} + 1}\right) - \frac{1}{2}\sin\left(\frac{\arcsin\left(\sqrt{p_I^{\min}}\right)}{2k^{\max} + 1}\right) \,. \tag{89}$$

Using Lemmas 1, 3 and 4, we obtain:

$$\begin{aligned}\epsilon_I^a(k^{\max}) &\leq \frac{1}{2}\sin\left(\frac{2\arcsin\left(\sqrt{2\epsilon_i^p}\right)}{2k^{\max}+1}\right)\\ &\leq \frac{1}{2}\sin\left(\frac{2\arcsin\left(\sqrt{2\epsilon^p(q,k^{\max})}\right)}{2\left\lceil\frac{\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)}-\frac{1}{2}\right\rceil+1}\right)\leq\epsilon\,,\end{aligned} \tag{90}$$

where we have also used property *i)* and the definition of $k^{\max}$. So, we have proven property *iii)*.

Proof of property iv) In this subsection we bound the maximum number of iterations needed to achieve the target accuracy ϵ. First note that, if I represents the last iteration, we have that

$$\epsilon < \epsilon_{I-1}^a = \frac{1}{2}\sin\left(\frac{\arcsin\left(\sqrt{p_{I-1}^{\max}}\right)}{2k_{I-1}+1}\right)-\frac{1}{2}\sin\left(\frac{\arcsin\left(\sqrt{p_{I-1}^{\min}}\right)}{2k_{I-1}+1}\right), \tag{91}$$

otherwise we would be in the last iteration, and that is false by hypothesis. In order to write (91) we have used (58) with $i = I-1$. Using similar arguments as in property *iii*, we bound ϵ_{I-1}^a by

$$\epsilon < \epsilon_{I-1}^a \leq \frac{1}{2}\sin\left(\frac{2\arcsin\left(\sqrt{2\epsilon_{I-1}^p}\right)}{2k_{I-1}+1}\right). \tag{92}$$

So, we can rewrite (92) as

$$\begin{aligned}(2k_1+1)\prod_{i=1}^{I-2}q_i &= 2k_{I-1}+1 < \frac{2\arcsin\left(\sqrt{2\epsilon_{I-1}^p}\right)}{\arcsin(2\epsilon)}\\ &\leq \frac{2\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)} =: (2k_1+1)\prod_{i=1}^{T-2}q = (2k_1+1)q^{T-2}\,,\end{aligned} \tag{93}$$

where we have used $\epsilon_i^p \leq \epsilon^p(q,\infty)$ and we have introduced the positive number T. Additionally, from (93), we obtain that

$$\prod_{i=1}^{I-2}q_i < q^{T-2}\,. \tag{94}$$

By using property *ii)*, we get

$$T = \log_q\left(q^2\frac{2\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)}\right) > I, \tag{95}$$

thus proving property *iv)*. *Proof of property v)* We aim to ensure that the precision ϵ is met with confidence $1-\gamma$. In order to achieve this, note that:

$$\begin{aligned}\mathbb{P}\Big[a \notin [a_I^{\min}, a_I^{\max}]\Big] &= \mathbb{P}\Big[\sin^2[(2k_I+1)\theta_I] \notin [p_I^{\min}, p_I^{\max}]\Big] \\ &\leq \mathbb{P}\Big[\bigcup_{i=1}^{I} \sin^2[(2k_i+1)\theta_i] \notin [p_i^{\min}, p_i^{\max}]\Big] \\ &\leq \sum_{i=1}^{I} \mathbb{P}\Big[\sin^2[(2k_i+1)\theta_i] \notin [p_i^{\min}, p_i^{\max}]\Big] \\ &\leq \sum_{i=1}^{I} \gamma_i = \frac{\gamma}{2}\frac{1}{2k^{\max}+1}\frac{q-1}{q} + \frac{\gamma}{2}\frac{1}{2k^{\max}+1}\frac{q-1}{q}\sum_{i=2}^{I} 2k_i+1,\end{aligned}$$

where we have used the definition of γ_i and Boole's inequality.

Next, applying Lemma 5 with $f(x) = x$ and $2k_{i+1}+1 \geq q(2k_i+1)$, we obtain:

$$\begin{aligned}\mathbb{P}\Big[a \notin [a_I^{\min}, a_I^{\max}]\Big] &\leq \frac{\gamma}{2}\frac{1}{2k^{\max}+1}\frac{q-1}{q} + \frac{\gamma}{2}\frac{q-1}{q}\frac{1}{2k^{\max}+1}\sum_{i=1}^{I-1}\frac{2k^{\max}+1}{q^i} \\ &\leq \gamma\frac{q-1}{q}\sum_{i=0}^{\infty}\frac{1}{q^i} = \gamma,\end{aligned}$$

where we have used the property *iii)*. We recall that $1-\gamma_i$ represents the confidence level of the i-th iteration.

Proof of property vi) We want to find an upper bound on the number of calls to the Grover oracle in order to obtain a target precision ϵ with confidence $1-\gamma$.

As we finish after I iterations, the number of calls to the Grover oracle is given by

$$N_Q = \sum_{i=1}^{I} N_i k_i < \frac{1}{2}\sum_{i=1}^{I} N_i(2k_i+1). \tag{96}$$

Next, we need an upper bound $\overline{N}_i$ for the number of shots N_i at each iteration:

$$\begin{aligned}N_i &= \left\lceil \frac{1}{2\epsilon^p(q,k_i)^2}\log\left(\frac{2}{\gamma_i}\right)\right\rceil \\ &< \frac{1}{2\epsilon^p(q,k_i)^2}\log\left(\frac{2}{\gamma_i}\right) + 1 < \frac{1}{2\epsilon^p(q,0)^2}\log\left(\frac{2\sqrt{e}}{\gamma_i}\right) =: \overline{N}_i,\end{aligned} \tag{97}$$

where we have redefined $\epsilon^p(q,0)$ with respect to Equation (63) as:

$$\epsilon^p(q,0) = \frac{1}{2}\sin^2\left(\frac{\pi}{4(q+2)}\right). \tag{98}$$

Replacing (97) in Equation (96) we get:

$$
\begin{aligned}
N_Q &< \frac{1}{2}\sum_{i=1}^{I}\overline{N}_i(2k_i+1) = \frac{1}{4\epsilon^p(q,0)^2}\sum_{i=1}^{I}\log\left(\frac{2\sqrt{e}}{\gamma_i}\right)(2k_i+1)\\
&= \frac{1}{4\epsilon^p(q,0)^2}\sum_{i=1}^{I}\log\left(\frac{4\sqrt{e}q(2k^{\max}+1)}{\gamma(q-1)(2k_i+1)}\right)(2k_i+1).
\end{aligned}
$$

Next, by using Lemma 5 on the function $x\,\log\left(\frac{c}{x}\right)$ we can find an upper bound of the sum:

$$
\begin{aligned}
N_Q &< \frac{1}{4\epsilon^p(q,0)^2}\sum_{i=1}^{I-1}\log\left(\frac{4\sqrt{e}q(2k^{\max}+1)q^i}{\gamma(q-1)(2k^{\max}+1)}\right)\frac{(2k^{\max}+1)}{q^i}\\
&= \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\sum_{i=1}^{I-1}\log\left(\frac{4\sqrt{e}q^{i+1}}{\gamma(q-1)}\right)\frac{1}{q^i}\\
&= \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\left(\log\left(\frac{4\sqrt{e}q}{\gamma(q-1)}\right)\sum_{i=1}^{I-1}\frac{1}{q^i}+\sum_{i=1}^{I-1}\log\left(q^i\right)\frac{1}{q^i}\right)\\
&= \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\left(\log\left(\frac{4\sqrt{e}q}{\gamma(q-1)}\right)\sum_{i=1}^{I-1}\frac{1}{q^i}+\log(q)\sum_{i=1}^{I-1}i\frac{1}{q^i}\right).
\end{aligned}
$$

Then, in order to upper bound the sums we extend the summation of positive numbers up to infinity:

$$
\begin{aligned}
N_Q &< \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\left(\log\left(\frac{4\sqrt{e}q}{\gamma(q-1)}\right)\sum_{i=1}^{I-1}\frac{1}{q^i}+\log(q)\sum_{i=1}^{I-1}i\frac{1}{q^i}\right)\\
&\leq \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\left(\log\left(\frac{4\sqrt{e}q}{\gamma(q-1)}\right)\sum_{i=0}^{\infty}\frac{1}{q^i}+\log(q)\sum_{i=0}^{\infty}i\frac{1}{q^i}\right)\\
&\leq \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\left(\log\left(\frac{4\sqrt{e}q}{\gamma(q-1)}\right)\frac{q}{q-1}+\log(q)\frac{q}{(q-1)^2}\right)\\
&= \frac{2k^{\max}+1}{4\epsilon^p(q,0)^2}\frac{q}{q-1}\log\left(\frac{4\sqrt{e}q^{\frac{q}{q-1}}}{\gamma(q-1)}\right)\leq\\
&\leq \left(2\frac{\arcsin\left(\sqrt{2\epsilon^p(q,\infty)}\right)}{\arcsin(2\epsilon)}+2\right)\frac{1}{4\epsilon^p(q,0)^2}\frac{q}{q-1}\log\left(\frac{4\sqrt{e}q^{\frac{q}{q-1}}}{\gamma(q-1)}\right)\\
&\leq \left(\frac{1}{\arcsin(2\epsilon)}\right)\frac{\pi}{2\epsilon^p(q,0)^2}\frac{q}{q-1}\log\left(\frac{4\sqrt{e}q^{\frac{q}{q-1}}}{\gamma(q-1)}\right)\\
&\leq C_1(q)\frac{1}{\epsilon}\log\left(\frac{C_2(q)}{\gamma}\right),
\end{aligned}
$$

and we end the proof of property *vi)*. □

Acknowledgements
The authors would like to thank Vedran Dunjko and Emil Dimitrov for fruitful discussions on some aspects of the present work. The authors also acknowledge Galicia Supercomputing Center (CESGA) for providing access to FinisTerrae III

supercomputer with financing from the Programa Operativo Plurirregional de España 2014-2020 of ERDF, ICTS-2019-02-CESGA-3.

Author contributions
A.M. wrote the main manuscript. G.F. performed the experiments. All authors provided ideas related to the different aspects of the work (financial and quantum computing theoretical, practical and implementation issues) and reviewed the manuscript.

Funding
All authors acknowledge the European Project NExt ApplicationS of Quantum Computing (NEASQC), funded by Horizon 2020 Program inside the call H2020-FETFLAG-2020-01 (Grant Agreement 951821). Á. Leitao, A. Manzano and C. Vázquez also acknowledge funding from the Galician Government (grant ED431C 2022/47, including FEDER financial support). Á. Leitao, A. Manzano and C. Vázquez also acknowledge the support from CITIC, as a center accredited for excellence within the Galician University System and a member of the CIGUS Network, receives subsidies from the Department of Education, Science, Universities, and Vocational Training of the Xunta de Galicia. Additionally, it is co-financed by the EU through the FEDER Galicia 2021-27 operational program (Ref. ED431G 2023/01).

Data Availability
No datasets were generated or analysed during the current study.

Declarations

Competing interests
The authors declare no competing interests.

Author details
[1] Department of Mathematics and CITIC, University of A Coruña, Campus Elviña s/n, A Coruña, 15071, A Coruña, Spain.
[2] Applications and Projects Department, Galicia Supercomputing Center (CESGA), Avenida de Vigo, Santiago de Compostela, 15705, A Coruña, Spain.

Received: 17 July 2024 Accepted: 10 February 2025 Published online: 25 February 2025

References
1. Gómez A, Leitao A, Manzano A, Nogueiras M, Ordóñez G, Vázquez C. A survey on quantum computational finance for derivatives pricing and var. Arch Comput Methods Eng. 2022.
2. Glasserman P. Monte Carlo methods in financial engineering. New York: Springer; 2004.
3. Montanaro A. Quantum speedup of Monte Carlo methods. Proc R Soc A, Math Phys Eng Sci. 2015;471(2181):20150301.
4. Rebentrost P, Gupt B, Bromley TR. Quantum computational finance: Monte Carlo pricing of financial derivatives. Phys Rev A. 2018;98(2).
5. Stamatopoulos N, Egger DJ, Sun Y, Zoufal C, Iten R, Shen N, Woerner S. Option pricing using quantum computers. Quantum. 2020;4:291.
6. Stamatopoulos N, Zeng WJ. Derivative pricing using quantum signal processing. Quantum. 2024;8:1322. https://doi.org/10.22331/q-2024-04-30-1322.
7. Ghysels E, Morgan J, Mohammadbagherpoor H. Quantum Computational Algorithms for Derivative Pricing and Credit Risk in a Regime Switching Economy. 2023. https://arxiv.org/abs/2311.00825.
8. Udvarnoki Z, Fáth G, Fogarasi N. Quantum advantage of Monte Carlo option pricing. J Phys Commun. 2023;7(5):055001. https://doi.org/10.1088/2399-6528/acd2a4.
9. Manzano A, Musso D, Leitao Á. Real quantum amplitude estimation. EPJ Quantum Technol. 2023;10(1):2.
10. Hull J. Options, futures, and other derivatives. Boston: Prentice Hall; 1997.
11. Kjaer M. On the pricing of Cliquet Options with global floor and cap. Master's thesis, Department of Mathematics, Chalmers University Goteborg; 2004.
12. Achdou Y, Pironneau O. Computational methods for option pricing. Philadelphia: SIAM; 2005.
13. Kloeden PE, Platen E. Numerical solution of stochastic differential equations. vol. 23. New York: Springer; 2013.
14. Grover L, Rudolph T. Creating superpositions that correspond to efficiently integrable probability distributions. 2002. arXiv.
15. Shende VV, Bullock SS, Markov IL. Synthesis of quantum-logic circuits. IEEE Trans Comput-Aided Des Integr Circuits Syst. 2006;25(6):1000–10.
16. Nielsen MA, Chuang IL. Quantum computation and quantum information. Cambridge: Cambridge University Press; 2000.
17. Brassard G, Hoyer P, Mosca M, Tapp A. Quantum amplitude amplification and estimation. Contemp Math. 2002;305:53–74.
18. List of quantum processors. Wikipedia. 2024. https://en.wikipedia.org/wiki/List_of_quantum_processors.
19. Black F, Scholes M. The pricing of options and corporate liabilities. J Polit Econ. 1973;81(3):637–54.
20. Fukuzawa S, Ho C, Irani S, Zion J. Modified iterative quantum amplitude estimation is asymptotically optimal. In: 2023 proceedings of the symposium on algorithm engineering and experiments (ALENEX). Florence: SIAM; 2023. p. 135–47.
21. Suzuki Y, Uno S, Raymond R, Tanaka T, Onodera T, Yamamoto N. Amplitude estimation without phase estimation. Quantum Inf Process. 2020;19:1–17.
22. Giurgica-Tiron T, Kerenidis I, Labib F, Prakash A, Zeng W. Low depth algorithms for quantum amplitude estimation. Quantum. 2022;6:745.

23. Callison A, Browne DE. Improved maximum-likelihood quantum amplitude estimation. 2022. arXiv.
24. Grinko D, Gacon J, Zoufal C, Woerner S. Iterative quantum amplitude estimation. npj Quantum Inf. 2021;7(1).
25. Abrams DS, Williams CP. Fast quantum algorithms for numerical integrals and stochastic processes. 1999. arXiv.
26. Nakaji K. Faster amplitude estimation. Quantum Inf Comput. 2020;20(13&14):1109–23.
27. Zhao Y, Wang H, Xu K, Wang Y, Zhu J, Wang F. Adaptive algorithm for quantum amplitude estimation. 2022. arXiv.
28. Ferro G, Manzano A. NEASQC: Financial Applications. GitHub. 2024. https://github.com/NEASQC/FinancialApplications.
29. Manzano A, Musso D, Leitao A, Gómez A, Vázquez C, Ordóñez G, Nogueiras M. A modular framework for generic quantum algorithms. Mathematics. 2022;10:785.
30. Hoeffding W. Probability inequalities for sums of bounded random variables. J Am Stat Assoc. 1963;58(301):13–30.
31. Clopper CJ, Pearson ES. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika. 1934;26(4):404–13.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Submit your manuscript to a SpringerOpen journal and benefit from:

- ► Convenient online submission
- ► Rigorous peer review
- ► Open access: articles freely available online
- ► High visibility within the field
- ► Retaining the copyright to your article

Submit your next manuscript at ► springeropen.com